ISCY HANDBUCH

Version: Arbeitsstand Oktober 2026 (ISCY V23.7.34 developmentunreleased; letzter Stable Release V23.7.33 / Rust 0.3.22)

Dieses Handbuch erklaert ISCY fachlich und in einfacher Sprache. Es ist fuer Menschen geschrieben, die nicht aus einem ISMS-, Compliance- oder Informationssicherheits-Umfeld kommen.

ISCY ist eine selbst gehostete, lokale und datenschutzbewusste Open-Source-Plattform unter AGPL-3.0-only. Die Rust-/Axum-Anwendung wird aktiv entwickelt und befindet sich in einer fruehen Community-Adoptionsphase; Schnittstellen, Datenmodelle und Betriebsverfahren koennen sich weiterentwickeln.

ISCY wurde in dieser Codebasis mit Unterstuetzung von OpenAI Codex entwickelt, nach Rust migriert und technisch/fachlich plausibilisiert. Die fachliche Ausrichtung wurde mit offiziellen EU-Quellen zu NIS2, der NIS2-Durchfuehrungsverordnung (EU) 2024/2690, DORA, Cyber Resilience Act und EU AI Act sowie mit gaengigen ISMS-, Product-Security-, CVE-/SBOM-/CSAF-, Evidence- und Incident-Response-Praktiken abgeglichen. Das ist keine unabhaengige fachliche Pruefung, externe Zertifizierung oder Rechtsberatung. Regulatorische Unterstuetzung muss fuer Organisation, Rechtsraum und Einsatzkontext eigenstaendig bewertet werden.

Der Development-Stand ist keine vollstaendige Funktions-, Security- oder Produktionsfreigabe. Fachliche Begriffe und historische Ausbaustufen sind von tatsaechlich vorhandenen Rust-Schreibablaeufen zu unterscheiden. Insbesondere ist / derzeit eine Linkuebersicht. Die Migrationsluecke beim Assessment-Wizard besteht weiterhin (5.2). Die eigenstaendige SoA-Erzeugung und Control-Bearbeitung sind mit einer verbindlichen DRAFT-Schreibgrenze wiederhergestellt; ein Nachfolgeversions-/Freigabeablauf fehlt noch (5.10). Das bedeutet keine Aufhebung der weitergehenden bestehenden Produktanforderungen. Die historischen Evidence-Ausbaustufen in 6.12 bis 6.15 werden durch den Runtime-Stand in 6.16 ergaenzt und sind keine alternative aktuelle Support-Matrix.

Inhalt:

1. Was ISCY ist
2. Grundprinzipien in einfacher Sprache
3. Wie man ISCY fachlich lesen sollte
4. Navigation und Funktionsbereiche
5. Fachliche Erklaerung aller Hauptfunktionen
6. Typische Arbeitsablaeufe
7. Was die wichtigsten Begriffe bedeuten
8. Was ISCY ausdruecklich nicht ersetzt
9. Empfehlungen fuer die Einfuehrung
10. Strategische Weiterentwicklung
11. Git-Bezug dieses Handbuchs

1. Was ISCY ist:

ISCY ist eine Arbeitsplattform fuer:

- den Aufbau und die Pflege eines ISMS nach ISO 27001
- die Einordnung regulatorischer Anforderungen wie NIS2 und KRITIS
- die Planung, Bewertung und Nachverfolgung von Risiken
- die Dokumentation von Nachweisen, Audits und Management Reviews
- die strukturierte Bearbeitung von Produkt- und Software-Sicherheitsfragen
- die Bewertung und Korrelation von Schwachstellen und CVEs mit nativer Vulnerability Intelligence sowie einem optionalen Rust-LLM-Stub fuer strukturierte CVE-Zusammenfassungen; ein echtes Modell oder RAG wird im aktuellen Stand nicht ausgefuehrt
- die Auswertung von Zero-Trust-Agent-Posture fuer Windows, macOS und Linux

ISCY ist damit kein reines Ticketsystem und kein reines DMS. Es verbindet Governance, Nachweise, Risiken, Umsetzungsplanung und Produkt-Sicherheit in einem gemeinsamen Arbeitsmodell.

2. Grundprinzipien in einfacher Sprache:

2.1 Tenant:

Ein Tenant ist in ISCY die organisatorische Einheit, fuer die gearbeitet wird. Das kann zum Beispiel sein:

- ein Unternehmen
- eine Gesellschaft
- eine Business-Plattform
- ein Mandant in einer Beratungs- oder Gruppenstruktur

Fast alle Daten in ISCY gehoeren zu genau einem Tenant. Dadurch bleiben Daten voneinander getrennt.

2.2 Assessment Session:

Eine Assessment Session ist ein strukturierter Bewertungsdurchlauf. In ihr werden Profil, Scope, regulatorische Betroffenheit, Reifegrad und Ergebnisse zusammengefasst.

2.3 Requirement:

Ein Requirement ist eine Anforderung. Das kann eine ISO-27001-Anforderung, eine NIS2-bezogene Pflicht oder eine interne Vorgabe sein.

2.4 Evidence:

Evidence ist ein Nachweis. Zum Beispiel:

- eine Richtlinie
- ein Screenshot
- ein Auditprotokoll
- eine Verfahrensanweisung
- ein Export aus einem Drittsystem

2.5 Risk:

Ein Risk beschreibt ein moegliches negatives Ereignis fuer das Unternehmen. In ISCY wird ein Risiko ueber Auswirkung und Eintrittswahrscheinlichkeit bewertet.

2.6 Roadmap:

Die Roadmap uebersetzt Analyseergebnisse in konkrete Umsetzungsarbeit. Sie beantwortet die Frage: Was tun wir wann und warum?

2.7 CVE:

Eine CVE ist eine bekannte Schwachstelle mit standardisierter Kennung, zum Beispiel CVE-2026-12345.

2.8 Zero-Trust Agent:

Der Zero-Trust Agent ist ein read-only Collector. Er meldet Inventar, Heartbeats und Posture-Findings an ISCY. Produktive Agenten koennen mit Enrollment-Token aufgenommen werden und melden danach mit einem Agent-Secret, optional gebunden an einen mTLS-Client-Zertifikat-Fingerprint. Daraus entstehen keine automatischen Systemaenderungen, sondern nachvollziehbare Sichtbarkeit fuer Assets, Risiken, Evidenzen und Roadmap-Arbeit.

3. Wie man ISCY fachlich lesen sollte:

ISCY folgt fachlich einem roten Faden:

1. Organisation verstehen
2. Scope und Relevanz bestimmen
3. Anforderungen und Reifegrad bewerten
4. Risiken ableiten
5. Nachweise sammeln
6. Massnahmen planen
7. Audits, Reviews und Reports durchfuehren
8. Produkt- und Schwachstellen-Themen integrieren

4. Navigation und Funktionsbereiche:

Die wichtigsten Bereiche sind:

- Start-/Linkuebersicht unter /
- Guidance Navigator unter /navigator/
- Dashboard unter /dashboard/
- Zero Trust unter /zero-trust/

- Catalog unter /catalog/
- Reports unter /reports/
- Roadmap unter /roadmap/
- Evidence unter /evidence/
- Assets unter /assets/
- Suppliers unter /suppliers/
- Supplier/Product Security unter /suppliers/product-security/
- Imports unter /imports/
- Processes unter /processes/
- AI Governance unter /ai-governance/
- Requirements unter /requirements/
- Risks unter /risks/
- Assessments unter /assessments/
- Organizations unter /organizations/
- Product Security unter /product-security/
- Vulnerability Intelligence unter /cves/
- Threat Intelligence und Security Observations unter /security-observations/

5. Fachliche Erklaerung aller Hauptfunktionen:

5.1 Organizations:

Zweck:
Die Stammdaten des Mandanten erfassen.

Was hier gepflegt wird:

- Name und Identitaet des Unternehmens oder Mandanten
- Land und Einsatzlaender
- Sektor
- Groessenindikatoren wie Mitarbeitendenzahl und Umsatz
- kritische Dienstleistungen
- Relevanz fuer NIS2 oder KRITIS
- DORA-Rolle, also Finanzunternehmen oder IKT-Drittdienstleister
- DSGVO-Rolle, also Verantwortlicher, Auftragsverarbeiter und besondere Datenkategorien
- CRA-Relevanz fuer digitale Produkte
- AI-Act-Profil und Hochrisiko-Hinweis
- TISAX-Scope und ISO-27001-Zielbild
- regulatorische Notizen fuer Scope- und Managemententscheidungen
- Product-Security-Kontext

Fachlicher Nutzen:

- Grundlage fuer regulatorische Einordnung
- Grundlage fuer spaetere Berichte und Filter
- Entscheidungshilfe fuer NIS2-/KRITIS-Betroffenheit
- zentrale Vorsteuerung fuer DORA, DSGVO, CRA, AI Act, TISAX und ISO-27001-Arbeit

Technischer Stand:
Schreibberechtigte Nutzer koennen dieses Profil direkt in /organizations/ pflegen. Die Seite zeigt eine regulatorische Matrix mit aktiven Pfaden, Begruendung und naechsten fachlichen Schritten. Die API stellt dasselbe Profil ueber GET und PATCH /api/v1/organizations/tenant-profile bereit.

Fuer Nicht-Sicherheitsleute:
Dies ist die Stammdatenakte des Unternehmens in ISCY.

5.2 Wizard:

Zweck:
Den Nutzer Schritt fuer Schritt durch eine Erst- oder Wiederbewertung fuehren.

Schritte:

- Start
- Profil
- Applicability
- Scope
- Maturity
- Results

Der schreibende Wizard war in der frueheren Django-Anwendung implementiert:

WizardStartView, die vier Schrittansichten und WizardService.generateresults legten Sessions, Antworten, Bewertungen, Massnahmen, Reports und Roadmaps an. Der damalige Stand vor dem Rust-Cutover ist ueber den Elterncommit von b12db820ebac1c633873193eb7507ecc96d41319 nachvollziehbar.

Aktueller Integrationsbefund, keine geaenderte Produktanforderung:

Die Startseite / zeigt Linkkarten. Vorhandene Session-/Ergebnisdaten koennen ueber GET /api/v1/wizard/sessions und
GET /api/v1/wizard/sessions/{sessionid}/results gelesen werden, soweit der entsprechende Datenbestand vorhanden ist. Diese Pfade bieten keinen POST zur Anlage eines Wizard-Durchlaufs. Die Rust-Schreiblogik und Teile des benoetigten Schemas fehlen ebenfalls; allein das Eintragen eines POST-Handlers behebt diese Migrationsluecke nicht. Bis zur separat zu pruefenden Wiederherstellung ist der gefuhrte Bewertungsablauf nicht verfuegbar. Die Organisationsprofil-Pflege unter /organizations/ ist vorhanden, ersetzt den Wizard aber nicht.

Der gefuehrte Agent-Onboarding-Assistent unter /zero-trust/ ist eine andere, tatsaechlich implementierte Funktion und kein Assessment-Wizard.

5.3 Guidance Navigator:

Zweck:
Orientierung geben, welche fachlichen Schritte als naechstes sinnvoll sind.

Typische Inhalte:

- Arbeitspakete
- empfohlene Reihenfolge
- Status der Bearbeitung
- Detailseiten zu einzelnen Schritten

Fachlicher Nutzen:

- priorisierte Navigation
- weniger Leerlauf und weniger doppelte Arbeit
- bessere Anschlussfaehigkeit fuer neue Nutzer

Fuer Nicht-Sicherheitsleute:
Das ist die fachliche Landkarte durch das System.

5.4 Dashboard:

Zweck:
Management-taugliche Uebersicht ueber den aktuellen Stand.

Typische Inhalte:

- aggregierte Kennzahlen
- sektor- oder tenantbezogene Sicht
- Portfolio-PDF

Fachlicher Nutzen:

- schneller Statusblick
- Kommunikation an Leitung, Programmsteuerung oder Kunden

Fuer Nicht-Sicherheitsleute:
Das Dashboard ist das Cockpit.

5.5 Catalog:

Zweck:
Normative und fachliche Struktur sichtbar machen.

Typische Inhalte:

- Domaenen
- Struktur fuer Anforderungen
- fachliche Referenzbasis

Fachlicher Nutzen:

- gemeinsames Vokabular
- Vergleichbarkeit von Assessments

Fuer Nicht-Sicherheitsleute:
Der Catalog ist das Inhaltsverzeichnis der Bewertungslogik.

5.6 Requirements:

Zweck:
Anforderungen sichtbar und bearbeitbar machen.

Typische fachliche Fragen:

- Welche Pflicht existiert?
- Gilt sie fuer unseren Scope?
- Wie gut ist sie umgesetzt?
- Wo fehlt noch Nachweis?

Fachlicher Nutzen:

- Uebersicht ueber Soll-Anforderungen
- Bruecke zwischen Norm, Praxis und Nachweis

Fuer Nicht-Sicherheitsleute:
Hier steht, was man tun oder dokumentieren muss.

5.7 Processes:

Zweck:
Geschaeftsprozesse im Scope darstellen.

Typische Inhalte:

- Prozessname
- Scope
- Beschreibung
- Status
- Zuordnung zu Bereichen

Fachlicher Nutzen:

- Risiken und Anforderungen an echte Geschaeftsablaeufe anbinden
- Verantwortungen klarer machen

Fuer Nicht-Sicherheitsleute:
Ein Prozess ist ein geordneter Arbeitsablauf des Unternehmens.

5.8 Assets:

Zweck:
Informationswerte und wichtige Objekte dokumentieren.

Typische Asset-Beispiele:

- Anwendungen
- Datenbestaende
- Infrastruktur
- Services
- Dokumentationen

Fachlicher Nutzen:

- Risiken an konkrete Werte koppeln
- Kritikalitaet nachvollziehbar machen

Fuer Nicht-Sicherheitsleute:
Assets sind die Dinge, die fuer das Unternehmen wichtig sind und geschuetzt werden muessen.

5.8.1 Zero Trust:

Zweck:

Endpoint- und Infrastruktur-Posture aus Agenten sichtbar machen.

Typische Inhalte:

- registrierte Agent-Devices
- letzter Heartbeat
- Zero-Trust-Score
- Policy-Profile mit Sollbestand nach Tenant, OS, Asset-Typ, Business Unit oder Deployment-Channel
- gefuehrtes Agent-Onboarding fuer Windows, Linux, macOS und NixOS
- Enrollment-Token mit Ablauf, begrenzter Verwendung, Widerruf und Auditspur
- Soll-/Ist-Coverage, Heartbeat-Freshness und Finding-Grenzwerte je Policy
- Agent-PKI-/CSR-Governance mit CA-Provider-Metadaten, CSR-Review, Zertifikatsstatus, mTLS-Bindung, Rotation und Widerruf als Metadata-only-Modell
- naechster fachlicher Fokus aus Score, Severity und Agent-Freshness
- offene Findings nach Pillar und Severity
- Check-Katalog fuer Windows, macOS und Linux

Fachlicher Nutzen:

- technische Posture mit ISMS-Arbeit verbinden
- Findings in Risiken, Evidenzen und Roadmap-Arbeit ueberfuehren
- Zero-Trust-Optimierung nachvollziehbar und auditierbar machen
- Prioritaeten schneller erkennen, ohne Rohdaten manuell vergleichen zu muessen
- Agent-Abdeckung, veraltete Heartbeats und kritische Findings in der Betriebszentrale ueberwachen
- Policy-Abweichungen ueber sichere Webhooks aktiv zustellen und in der Delivery-Historie auditieren
- Zertifikats-, mTLS-, Rotations- und Widerrufsluecken frueh als Governance-Aufgabe sichtbar machen, ohne produktive CA-Aktionen zu automatisieren

Fuer Nicht-Sicherheitsleute:
Der Bereich zeigt, welche Geraete welche Sicherheitsluecken oder Nachweise melden.

Was die Agenten aktuell testen koennen:

- Inventar: Hostname, OS-Familie, OS-Version, CPU-Architektur, Agent-Version und Deployment-Channel
- Heartbeat: ob ein registriertes Geraet noch regelmaessig mit ISCY spricht
- OS-Baseline: Betriebssystem- und Patch-/Versionshinweise als Grundlage fuer MDM- oder Patch-Nachweise
- Datentraeger-Verschluesselung: BitLocker unter Windows, FileVault unter macOS, LUKS/root encryption unter Linux
- Plattformintegritaet: Secure Boot unter Windows/Linux sowie SIP/authenticated root unter macOS
- Host-Firewall: Windows-Firewall-Profile, macOS Application Firewall, Linux firewalld/ufw/nftables/iptables-Signale
- MDM oder Endpoint Management: Windows Enrollment Registry, macOS profiles-Enrollment, Linux-Management-Agenten wie osquery, Puppet, Chef, Salt, SSM oder vergleichbare Agenten
- Endpoint Protection/EDR: Windows Defender Status, macOS EDR-/Security-Agent-Pfade, Linux-Dienste und Pfade fuer Wazuh, auditd, Microsoft Defender, CrowdStrike, SentinelOne, osquery und vergleichbare Agenten

Die Agenten arbeiten read-only. Wenn ein Signal nicht sicher bestaetigt werden kann, meldet ISCY eine offene Evidenzluecke statt einen erfundenen Nachweis.

Nach dem ersten tokenbasierten Enrollment speichert der Agent Device-ID und Agent-Secret lokal. Unter Unix sind State-Verzeichnis und State-Datei mit 0700 beziehungsweise 0600 geschuetzt; der Windows-Installer setzt restriktive ACLs. Temporaer nicht zustellbare Reports werden in einer standardmaessig auf 100 Eintraege begrenzten Offline-Queue gepuffert und beim naechsten Lauf zuerst uebertragen. Admins koennen das Secret ueber POST /api/v1/agents/devices/{deviceid}/rotate-secret rotieren; das neue Secret wird nur einmal ausgegeben und muss ueber den sicheren Deployment-Kanal zum Endpoint gelangen.

Administratoren koennen in /zero-trust/ ueber Agent hinzufuegen einen dreistufigen Assistenten starten. Zuerst werden Betriebssystem, Deployment-Kanal, optionale Rollout-Bezeichnung, Policy-Profil, Nutzungsgrenze, Ablaufzeit und optionale mTLS-Bindung ausgewaehlt. Danach zeigt ISCY eine Zusammenfassung. Erst nach der Bestaetigung wird das Token erzeugt und genau einmal zusammen mit einer Anweisung fuer die vorhandenen Windows-, systemd-, macOS- oder NixOS-Artefakte

angezeigt.

Token-Klartexte werden nicht gespeichert. Die Metadatenansicht zeigt nur Hint, Status, Grenzen, Policy, Zaehler und Zeitpunkte. Die Zustaende sind pending, partiallyused, consumed, expired und revoked. Token-Verbrauch, Device-Aufnahme, Policy-Zuordnung, Secret-Hash und Auditereignis werden gemeinsam in einer Transaktion gespeichert. Read-only-Rollen duerfen sichere Metadaten sehen; Erstellen, Widerrufen und die einmalige Installationsansicht bleiben Administratoren vorbehalten.

Policy-Profile legen den erwarteten Device-Bestand, das maximale Heartbeat-Alter, den Mindestscore und tolerierte High-/Critical-Findings fest. Administratoren koennen Webhook-Kanaele mit Warnstufe und Cooldown konfigurieren. Bearer- und HMAC-Secrets werden dabei nur ueber Environment-Variablennamen referenziert und nicht in der Datenbank gespeichert. Im Production-Modus sind ausschliesslich explizit erlaubte Zielhosts zulaessig; Redirects werden nicht verfolgt. Jeder Versuch bleibt mit Ergebnis, HTTP-Status und Payload nachvollziehbar.

Die Weboberflaeche stellt Zero Trust bewusst als Arbeitsansicht dar:

- Score-Karte fuer die aktuelle Zero-Trust-Reife
- Fokuskarte fuer den naechsten sinnvollen Schritt
- Severity-Badges fuer kritische, hohe, mittlere und niedrige Findings
- mobile Navigation und horizontal scrollbare Tabellen fuer kleine Displays
- konservative Darstellung ohne automatische Remediation

5.9 Risks:

Zweck:
Risiken erfassen, bewerten und behandeln.

Kernlogik:

- Beschreibung des Risikos
- Bedrohung
- Schwachstelle
- Impact
- Likelihood
- Risikostufe
- Behandlungsstrategie
- Fristen und Reviews
- Review-Workflow fuer Behandlung, Akzeptanz, Mitigation und Abschluss
- automatische CVE-Risiken aus akzeptierten Product-Security-Korrelationen

Fachlicher Nutzen:

- zentrales Risikoregister
- transparente Priorisierung
- Nachvollziehbarkeit von Entscheidungen
- fachlicher Review von automatisch erzeugten Schwachstellenrisiken

Fuer Nicht-Sicherheitsleute:
Ein Risiko ist in ISCY kein Technikfehler, sondern eine moegliche negative Geschaeftsauswirkung.

5.10 Assessments:

Zweck:
Umsetzungs- und Nachweisstand strukturiert dokumentieren.

Teilfunktionen des bestehenden Assessment-Modells:

- Applicability
fachliche Frage: Gilt eine Anforderung fuer uns?
- Measures
fachliche Frage: Welche Massnahme setzen wir um?
- Statement of Applicability (SoA)
fachliche Frage: Welche ISO-27001 Controls sind relevant, umgesetzt oder ausgeschlossen?
- Audits
fachliche Frage: Was wurde geprueft und was wurde gefunden?
- Findings
fachliche Frage: Welche Abweichungen oder Schwachstellen wurden festgestellt?

- Management Reviews
fachliche Frage: Wie bewertet die Leitung den Gesamtstand?
- Review Actions
fachliche Frage: Welche Folgeaktionen beschliesst die Organisation?

Aktueller Rust-Umfang und Grenzen:

- /api/v1/assessments, /api/v1/assessments/applicability und /api/v1/assessments/measures sind GET-Leseregister, keine Assessment-Schreib-API. Nach reiner Rust-Demo-Initialisierung sind dort gegenwaertig Datenbankfehler bekannt; die Register gelten damit nicht als vollstaendig abgenommener Bewertungsablauf.
- Die fruehere Django-Anwendung bot unter POST /assessments/soa/generate/ eine SoA-Erzeugung aus aktiven ISO27001-Requirements an, mit Detailansicht und anschliessender Bearbeitung je Control. SoAGenerateView, SoADocument und SoAEntry sind im Elterncommit des oben genannten Rust-Cutovers belegt. Die eigenstaendige Erzeugung, Detailansicht und Control-Bearbeitung sind in Rust wiederhergestellt: Webliste unter /assessments/soa/, Erzeugung ueber POST /assessments/soa/generate/, API-Liste/Erzeugung ueber GET/POST /api/v1/assessments/soa und Control-Bearbeitung ueber PUT /api/v1/assessments/soa/entries/{id}. Migration 0047rustsoacutoverrestoration ergaenzt das benoetigte Schema und laesst bestehende Dokumente und Eintraege unveraendert. Neue Erstentwuerfe enthalten aktive ISO27001-Requirements als anwendbare Controls mit NOTSTARTED. Das ist keine automatische Konformitaetsentscheidung und kein impliziter Wizard-Abschluss.
- Nur der exakt gespeicherte Status DRAFT ist bearbeitbar. INREVIEW, APPROVED und unbekannte Statuswerte sind auch fuer Administratoren schreibgeschuetzt; die Ablehnung laesst Inhalte, Freigaben und Auditspur unveraendert. Statusentscheidung, Control-Aenderung und verpflichtender Auditeintrag erfolgen in einer gemeinsamen Transaktion. Rollen- und Tenant-Grenzen gelten fuer API und Web; Webformulare benoetigen zusaetzlich einen sessiongebundenen CSRF-Token. Das Import-Center bietet keinen SoA-Import an und weist SoA-Aliasse auch bei Replace-Importen zurueck.
- Aenderungen an geprueften/freigegebenen Dokumenten erfordern fachlich einen neuen, eindeutig zugeordneten Entwurf mit erneuter Pruefung/Freigabe. Dieser Nachfolgeprozess, historische Inhaltssnapshots und die Vorgaengerzuordnung sind noch nicht implementiert. Die vorhandene Erzeugung ist nur ein neuer unabhaengiger Erstentwurf und darf nicht als Nachfolgeversion eines gesperrten Dokuments verwendet werden. Die sichere Schreibsperre ersetzt keinen vollstaendigen Versionsworkflow; Details in docs/SOARESTORATIONCONTRACT.md.
- Management Reviews und Regulatory Review-Pakete aus 5.12 besitzen eigene Rust-Erzeugungs-, Review- und Exportpfade. Sie ersetzen keinen fehlenden Assessment-Wizard.

Fachlicher Nutzen:

- Kernbereich fuer ISMS-Nachweisfuehrung
- interne und externe Auditvorbereitung
- Management-Einbindung

Fuer Nicht-Sicherheitsleute:
Hier wird aus "wir glauben, dass wir gut sind" ein belastbarer und pruefbarer Nachweis.

5.11 Evidence:

Zweck:
Nachweise sammeln, pflegen und mit Bedarfen verknuepfen.

Typische Funktionen:

- Evidence hochladen
- Evidence aktualisieren
- Evidence Needs synchronisieren
- Evidence direkt aus Risks, Roadmap-Tasks, Incidents und Product-Security-Kontexten vorbefuellen
- nach dem Speichern automatisch zur Ausgangsseite zurueckkehren
- Evidence-Qualitaet unter /evidence/quality/ auswerten
- Score, Reifegrad und Issues fuer Nachweise und Evidence Needs anzeigen
- Datei-Uploads automatisch mit SHA-256 kennzeichnen
- Nachweise ueber Vorgaenger-ID und serverseitige Versionsnummer versionieren
- Gueltigkeit, Aufbewahrungsfrist, Retention-Begruendung und Schutzklasse pflegen

- abgelaufene und innerhalb von 30 Tagen ablaufende Nachweise in Quality-Queue und Betriebszentrale erkennen
- Version, Schutzklasse, Gueltigkeit und SHA-256 in Incident-/Regulatory-Exporten ausweisen
- Evidence-Integritaet unter /evidence/integrity/ pruefen
- vorhandene Evidence-Artefakte manuell oder begrenzt im Batch serverseitig neu mit SHA-256 hashen
- Integritaetsstatus, letzte Pruefung, berechneten Hash, Mismatch, sichere Fehlerklasse und Quarantaene-/Review-Status anzeigen
- Integritaets-Worker-Status, letzte Laeufe, Batch-Grenzen und empfohlene naechste Pruefung sehen
- Evidence-Integritaetspruefungen manuell als begrenzten Worker-Lauf mit Dry-Run-Option starten
- Legal Hold mit Begruendung setzen oder freigeben
- Disposition-/Retention-Entscheidungen als Governance-Metadaten dokumentieren
- Disposition-Kandidaten per Preview pruefen, freigeben, abbrechen oder nach Freigabe kontrolliert physisch aussondern
- Tombstone-Metadaten fuer kontrollierte physische Aussonderung erhalten, ohne Rohpfade oder Dateiinhalte offenzulegen
- Integritaets-, Legal-Hold- und Disposition-Ereignisse auditierbar nachvollziehen
- lokale Evidence-Artefakte ueber eine interne Storage-Abstraktion pruefen
- Storage-/Restore-Drills fuer Vorhandensein, Lesbarkeit und SHA-256-Konsistenz ausloesen
- Storage-Backend-Status fuer localfilesystem und vorbereitetes s3compatible ohne echte Cloud-Credentials auswerten
- Object-Storage-Backend-Metadaten, Secret-Referenzen, Endpoint-Policy, Bucket und Prefix tenantgebunden verwalten
- Object-Storage-Referenzen sicher an Evidence binden, ohne vollstaendige Object-Keys, Secretwerte oder Objektinhalte zu speichern
- Contract-basierte Object-Storage-Drills fuer vorhandene, fehlende, unlesbare, zeitueberschrittene oder Hash-abweichende Objekte dokumentieren
- sichere Storage-Fehlerklassen sehen, ohne absolute Dateipfade oder Rohpayloads offenzulegen

Fachlicher Nutzen:

- Nachweisfuehrung an einem Ort
- bessere Auditfaehigkeit
- weniger Suche nach Dokumenten
- klarere Rueckverfolgbarkeit durch stabile Linked-Requirement- und Evidence-Key-Bezuege
- belastbarere Aussage, ob Nachweise nur vorhanden oder wirklich reviewt und verwertbar sind
- nachvollziehbare Integritaet und Lifecycle-Steuerung ohne pauschal fest codierte gesetzliche Aufbewahrungsfrist
- klare Trennung zwischen Nachweisqualitaet, Integritaetspruefung, Legal Hold, metadata-only Disposition und kontrollierter physischer Aussonderung
- belastbarer Restore-/Integrity-Nachweis fuer lokal referenzierte Artefakte, ohne produktives Cloud-Storage vorauszusetzen

Fuer Nicht-Sicherheitsleute:
Evidence ist der Ordner mit den Belegen, aber strukturiert und auswertbar.

5.12 Reports:

Zweck:
Ergebnisse in lesbare und versendbare Form bringen.

Ausgaben:

- Report-Detailseite
- einfaches PDF
- audit-faehiges PDF
- Management-Review-Pakete unter /management-reviews/
- kontextsensitive Regulatory Review-Pakete unter /regulatory-review-packs/
- Management-Review-Exporte als Markdown, HTML, PDF und JSON

Management-Review-Pakete:

- werden aus aktuellen ISCY-Daten fuer einen Zeitraum erzeugt
- koennen als generisches Security-Governance-Paket oder ueber Templates fuer ISO 27001, NIS2, DORA, DSGVO und KRITIS vorbereitet werden
- speichern Top-Risiken, ISCY-27-Control-Gaps, Evidence-Luecken, Evidence-Integrity-/Storage-Aggregate, Incident-Entscheidungen, Roadmap-Fokus, Product-Security-Lage, Supplier-Review-Summary, Supplier/Product-Security-Lage, Agent-Posture, AI-Governance, Quellenzaehlung, Gap-Summary, Management-Hinweise und regulatorischen Kontext als Snapshot

- verlinken Snapshot-Zeilen zurueck zu Risiko, Control, Evidence, Incident und Roadmap
- unterstuetzen die Statuswechsel Draft, In Review und Approved; ein Archived-Statuswechsel wird vom aktuellen Management-Review-Store nicht angeboten
- dokumentieren Entscheidung, naechste Massnahmen, freigebenden User und Freigabezeitpunkt
- erlauben eine Template-Vorschau, ohne bereits ein Review-Paket einzufrieren

Regulatory Review-Pakete:

- nutzen dieselbe eingefrorene Snapshot-Schicht wie Management Reviews und erzeugen kein separates Compliance-Silo
- bieten fokussierte Pack-Typen fuer NIS2, DORA und DSGVO sowie ein generisches Security-Governance-Pack
- beruecksichtigen Organisationsprofil, Risiken, ISCY-27-Controls, Evidence-Qualitaet, Evidence-Integritaet, Storage-/Restore-Drill-Signale, Incidents, Supplier Review, Supplier/Product Security, Product Security, AI Governance, Roadmap und Agent Posture soweit vorhanden
- unterscheiden Preview und Snapshot-Erzeugung: Read-only-Rollen koennen Vorschauen lesen, aber keine eingefrorenen Snapshots erstellen
- bieten sichere Filter fuer Pack-Typ, Status, Zeitraum, offene Luecken, kritische Luecken und Limit
- zeigen Owner-/Verantwortlichen-Hinweise aus vorhandenen Rollen-/Owner-Daten; fehlende Verantwortliche werden als Nicht erfasst markiert und nicht geraten
- gruppieren Luecken pack-spezifisch, z. B. NIS2 nach Incident-/Meldeentscheidungen, TOMs, Evidence und Supplier; DORA nach ICT-Risk, Incident, ICT-Third-Party und Storage; DSGVO nach Datenschutzrollen, Data-Breach, Legal Hold / Aufbewahrungssperre, Disposition und Supplier-Datenbezug
- nehmen Supplier/Product-Security-Gaps auf: offene Advisorys, kritische Lieferantenabhaengigkeiten, fehlende Evidence, fehlende Owner, offene Massnahmen, ueberfaellige Reviews sowie Vertrags-/Exit-Plan-Hinweise fuer DORA, NIS2 und DSGVO
- enthalten Hinweise, dass ISCY Governance- und Evidence-Unterstuetzung liefert, jedoch keine Rechtsberatung, Zertifizierung, automatische Meldung oder formale Einreichung ersetzt

API- und Web-Pfade:

- GET /api/v1/management/templates
- GET /api/v1/management/templates/{templatetype}
- POST /api/v1/regulatory/templates/{templatetype}/preview
- GET und POST /api/v1/management/reviews
- GET /api/v1/regulatory/review-packs
- GET /api/v1/regulatory/review-packs/{packtype}
- POST /api/v1/regulatory/review-packs/{packtype}/preview
- GET und POST /api/v1/regulatory/review-packs/{packtype}/snapshots
- GET /api/v1/regulatory/review-pack-snapshots mit optionalen Filtern packtype, status, periodstart, periodend, hasopengaps, hascriticalgaps und limit
- GET /api/v1/regulatory/review-pack-snapshots/{snapshotid}
- GET /api/v1/regulatory/review-pack-snapshots/{snapshotid}/export?format=markdown|html|pdf|json
- Weboberflaeche unter /management-reviews/ mit Template-Auswahl und Preview
- Weboberflaeche unter /regulatory-review-packs/ mit Pack-Auswahl, Filterbereich, Vorschau, Owner-Hinweisen, pack-spezifischer Lueckenuebersicht, Snapshot-Erzeugung und Snapshot-Liste

Fachlicher Nutzen:

- Management-Kommunikation
- Dokumentationsstand
- Vorlagen fuer Kunden, Auditoren oder interne Gremien
- belastbare Vorbereitung von Management Review, Audit und Steering Committee
- wiederholbare Steuerungspakete, ohne neue Risiko-, Evidence- oder Control-Silos anzulegen
- regulatorischer Kontext wird aus vorhandenen ISCY-Daten abgeleitet und bleibt als Snapshot nachvollziehbar
- deutsche UI-Beschriftungen und Empty States machen die Review-Pakete fuer Management, Fachbereich und Audit leichter nutzbar

GUI-Nachweis:

- Die aktuelle Screenshot-Uebersicht fuer Dashboard, Evidence, Regulatory Review-Pakete, Supplier Review, Supplier/Product Security, Product Security, AI Governance und Operations liegt in docs/GUI_SCREENSHOTS.md.

Fuer Nicht-Sicherheitsleute:
Reports sind die offizielle Zusammenfassung des Stands.

5.13 Roadmap:

Zweck:
Aus Ergebnissen konkrete Arbeit machen.

Typische Funktionen:

- Planliste
- Plandetail
- Kanban-Ansicht
- Task-Bearbeitung
- PDF- und PNG-Export
- direkte Evidence-Verknuepfung je Roadmap-Task

Fachlicher Nutzen:

- Transformation von Analyse in Umsetzungsprogramm
- Priorisierung nach Wirkung und Aufwand
- Nachweisfuehrung dort starten, wo die Umsetzungsarbeit entsteht

Fuer Nicht-Sicherheitsleute:
Die Roadmap ist der Umsetzungsfahrplan.

5.14 Import Center:

Zweck:
Bestehende Daten strukturiert nach ISCY uebernehmen.

Typische Funktionen:

- Import Guide
- Mapping Assistant
- Vorschau
- Template-Download

Importierbare Inhalte:

- Business Units
- Prozesse
- Lieferanten
- Assets

Sicherheitsgrenzen fuer Tabellenimporte:

- Uploads sind auf 12 MiB begrenzt; die Vorschau zeigt maximal 200 Zeilen.
- XLSX/XLSM-Dateien werden vor dem eigentlichen Parser als ZIP-/Office-Archiv geprueft. Maximal 512 Archiveintraege, 64 MiB gesamte entpackte Daten und 32 MiB pro Eintrag werden akzeptiert.
- Shared Strings sind auf 250.000 eindeutige Eintraege begrenzt; deklarierte Zaehler und die tatsaechlich gelesenen Elemente werden geprueft.
- Der XLSX-Import verarbeitet maximal 1.000.000 verwendete Zellen und 50.000 Importzeilen. Sparse Tabellen werden zellenweise gelesen, damit weit auseinanderliegende Zellkoordinaten nicht als riesiges Rechteck materialisiert werden.
- Ungueltige oder uebergrosse Archive werden fail-closed vor der eigentlichen Tabellenverarbeitung abgelehnt.

Diese Grenzen reduzieren Memory-/Decompression-DoS-Risiken, ersetzen aber keinen Malware-Scanner und keine Parser-Sandbox fuer besonders riskante Einsatzumgebungen.

Fachlicher Nutzen:

- schneller Projektstart
- weniger manuelle Datenerfassung
- bessere Datenqualitaet durch Mapping

Fuer Nicht-Sicherheitsleute:
Das Import Center ist die Uebersetzungsstelle fuer vorhandene Tabellen in das ISCY-Datenmodell.

5.15 Product Security:

Zweck:
Sicherheitsaspekte digitaler Produkte verwalten.

Typische Objekte:

- Product Family
- Product
- Product Release
- Component
- AI System
- Threat Model
- Threat Scenario
- TARA
- Vulnerability
- VEX-Entscheidung
- PSIRT Case
- CSAF-/SBOM-Importhistorie
- lokale Supplier-Advisory-/PSIRT-/CVE-Metadaten
- SBOM-Diff
- CVE-Asset-Korrelation
- CVE-Risiko-Review-Queue
- CRA-Readiness
- versioniertes Release-/PSIRT-Evidence-Paket

Fachlicher Nutzen:

- Anbindung von Produktentwicklung und Sicherheitsgovernance
- Vorbereitung fuer CRA, IEC 62443, ISO/SAE 21434 oder AI-bezogene Governance
- Sicht auf Releases, Komponenten und Verwundbarkeit
- strukturierte Verarbeitung von CSAF-Advisories, CycloneDX/SPDX-SBOMs und CVE-Korrelationen
- auditierbare VEX-Entscheidung je Schwachstelle mit Status, Begruendung, Fix-Version und Zeitpunkt
- SBOM-Vergleich zwischen Importstaenden, damit neue, entfernte und geaenderte Komponenten sichtbar werden
- CRA-Readiness je Produkt aus SBOM, VEX/CVE-Triage, PSIRT/Advisories, Threat/TARA und Lifecycle
- automatische Ableitung von Risiko- und Roadmap-Arbeit aus akzeptierten CVE-Korrelationen
- Nachweissteuerung ueber Evidence-Keys fuer CVE, Import, Risiko und Roadmap
- dokumentierte Release-/PSIRT-Entscheidungen mit eingefrorenem Nachweisstand und Blocker-Gates
- Verbindung zu Supplier/Product Security, damit Hersteller-, Lieferanten- und Produktbezuege gemeinsam mit Evidence, Review-Status und offenen Massnahmen sichtbar werden

Fuer Nicht-Sicherheitsleute:
Dieser Bereich ist fuer Unternehmen wichtig, die Software, digitale Produkte oder vernetzte Systeme bereitstellen.

5.15.1 Safety & Conformity

Der Product-Security-nahe Arbeitsbereich /product-safety/ verbindet Cybersecurity und funktionale Produktsicherheit, ohne beide in dasselbe Risikomodell zu pressen. Product Applicability fuer CRA und Maschinenverordnung, Maschinenprofil, Safety Functions, Hazards und qualitative Safety Assessments bleiben eigene fachliche Datensaetze. Typisierte Interactions zeigen explizit, wie eine vorhandene Vulnerability, Security Observation, ein Risk, eine SBOM-Komponente, CVE-Korrelation oder ein Threat/TARA-Kontext eine Safety Function oder einen Hazard beruehren kann.

Die Detailansicht trennt Cyber Source, Interaction und Safety optisch und zeigt Requirements, Evidence-Gaps, offene Reviews sowie eine technische Dokumentationsvorschau. Der allgemeine Anwendungstermin der Maschinenverordnung (EU) 2023/1230 ist gemaess dem offiziellen EUR-Lex-Korrigendum zu Artikel 54 der 20. Januar 2027.

Die Technical-Documentation-Readiness folgt einer fail-closed Matrix:

- EVIDENCEGAPS: Pflichtdaten, Requirement-Evidence oder Evidence-Links fehlen.
- ASSESSMENTINPROGRESS: Grunddaten und Evidence sind vorhanden, aber Hazards, Reviews oder MITIGATIONREQUIRED-Interactions sind offen.

- READYFORHUMANREVIEW: keine strukturellen Luecken und keine offenen fachlichen Blocker; die menschliche Abschlusspruefung bleibt erforderlich.

humanassessment bleibt immer REQUIRED. Die GUI stellt auch READYFORHUMANREVIEW als Warn-/Review-Zustand und nicht als gruene Endfreigabe dar. CLOSED bezeichnet nur das Ende eines Workflows. ISCY trifft keine Rechts-, CE-, Safety-, CRA-, MVO-, Norm- oder Konformitaetsentscheidung. Die fachliche Architektur, API und Quellen sind in MACHINERY_CRA_SAFETY_SECURITY.md dokumentiert.

Aktueller Rust-Funktionsumfang:

- Import-Historie fuer CSAF, CycloneDX und SPDX mit CSV-/JSON-Export
- Import-Detailseite mit Validierungsfehlern und Komponenten-Matches
- SBOM-Diff als Webansicht und API fuer zwei SBOM-Importartefakte
- VEX-Status fuer Schwachstellen: betroffen, nicht betroffen, behoben oder in Untersuchung
- CRA-Readiness-Dashboard je Produkt mit transparenten Dimensionen und Gap-Hinweisen
- CVE-Asset-Korrelation ueber CPE oder PURL mit Akzeptieren/Ablehnen-Workflow
- automatische Erzeugung von CVE-Risiken und Product-Security-Roadmap-Tasks aus akzeptierten Korrelationen
- Dashboard-Kennzahlen fuer offene CVE-Reviews und fehlende Evidence
- gebuendelte CVE-Risiko-Review-Queue mit Filtern fuer offene Reviews, fehlende Evidence und fehlende Risiken
- Bulk-Aktionen fuer ausgewaehlte CVE-Reviews: Risiko/Roadmap erzeugen, Behandlung freigeben, Restrisiko akzeptieren oder als mitigiert markieren
- Einzelaktionen fuer Behandeln, Akzeptieren und Mitigiert markieren
- Evidence-Vorbefuellung und Ruecksprung zur Ausgangsseite nach Upload
- Paketregister unter /product-security/evidence-packages/ fuer Release- und PSIRT-Scope
- versionierte Snapshots aus Lifecycle, SBOM, VEX, Advisories, Risiko-/Roadmap-Stand und Evidence
- Review-Workflow mit Entwurf, Review, Aenderungsanforderung, Freigabe und Archivierung
- vorbehaltlose Freigabe nur ohne Blocker; bedingte Freigabe nur mit dokumentierter Review-Notiz
- Markdown-, HTML-, PDF- und JSON-Export je Paketversion
- Betriebsstatus fuer Paket-Backlog und offene Blocker
- Supplier/Product-Security-Register unter /suppliers/product-security/ fuer lokale Advisory-/PSIRT-/CVE-Metadaten, Lieferantenprodukt, betroffene und behobene Versionen, SBOM-/VEX-Bezug, Review-Status, Owner, Faelligkeit und Evidence
- API-Pfade unter /api/v1/suppliers/product-security fuer Liste, Anlage, Detail, Aenderung, Statuswechsel, Evidence-Verknuepfung und Ereignishistorie
- Das Supplier/Product-Security-Register selbst fuehrt keine freien Live-Abfragen gegen Herstellerportale oder beliebige Advisory-URLs aus. Die zentrale, fest erlaubte Vulnerability-Intelligence-Synchronisation wird getrennt unter 5.18 betrieben; Supplier-Referenzen werden lokal sicher gespeichert und ausgegeben.

5.16 AI Governance:

Zweck:
KI-Systeme als eigene Governance-Objekte steuern.

AI Governance umfasst AI-Systeme, die in Produkten, internen Prozessen oder Support-/Triage-Flows genutzt werden. ISCY trennt dabei nicht nur nach Regulierung, sondern fragt funktional: Welchen Zweck hat das System, welche Daten nutzt es, welche Wirkung haben seine Empfehlungen oder Entscheidungen, wer prueft die Ergebnisse und welche Evidence belegt Betrieb, Review und Kontrolle?

Typische Objekte:

- AI System
- Produktbezug
- Modellquelle und Provider
- Datenkategorien
- Entscheidungswirkung
- Human Oversight
- AI-Act-Klassifizierung
- Monitoringplan
- Risikosummary
- Evidence-Key

Aktueller Rust-Funktionsumfang:

- Webansicht /ai-governance/ mit AI-Systemregister, Kennzahlen, Review-Faelligkeit, Evidence-Stand und Governance-Gaps
- API GET und POST /api/v1/ai-governance/systems
- API GET und PATCH /api/v1/ai-governance/systems/{id}
- API GET /api/v1/ai-governance/systems/{id}/link-candidates
- API POST und DELETE /api/v1/ai-governance/systems/{id}/links/{typ}/{objekt-id} fuer Risiken, Roadmap-Tasks, Incidents und Changes
- API POST /api/v1/ai-governance/systems/{id}/gap-tasks fuer die ausdruecklich ausgeloeste Task-Erzeugung
- AI-Act-Klassen: nicht bewertet, High Risk, Limited Risk, Minimal Risk, nicht im Scope und verboten/nicht freigegeben
- Anforderungen fuer Klassifizierung, Risikomanagement, Human Oversight, Logging, Transparenz, Cybersecurity/Robustheit sowie Monitoring/Evidence
- Evidence-Vorbefuellung ueber stabile AI-Governance-Evidence-Keys
- direkte, im SQL tenantgebundene Verknuepfungen mit bestehenden Risiken, Roadmap-Tasks und Incidents
- kleines allgemeines Change-Register als kanonischer Plattformkern; es ist kein AI-spezifisches Ersatzmodell und noch kein vollstaendiger Change-Management-Workflow
- Link-Audit fuer Anlegen und Entfernen mit Actor, Objekttyp, Objekt-ID und Zeitpunkt
- Roadmap-Tasks aus offenen Governance-Gaps nur nach Nutzeraktion; originkey und Unique-Index verhindern Duplikate auch bei Wiederholung
- Management-Review-Snapshots und Exporte frieren AI-Systeme sowie Linkzaehler fuer Risiko, Roadmap, Incident und Change ein; Management-/Regulatory-Templates nehmen diese Daten in den passenden Review-Kontext auf
- Rust-only-Betriebssignale fuer nicht bewertete AI-Systeme, faellige Reviews, fehlende Evidence und offene Governance-Gaps

Tenant- und Berechtigungsgrenzen:

- Leser und Auditoren koennen die Verknuepfungen sehen, aber nicht aendern.
- Schreibende Rollen duerfen Links und Gap-Tasks verwalten.
- AI-System und Zielobjekt werden bereits in der Datenbankabfrage auf denselben Tenant eingeschraenkt. Manipulierte oder fremde IDs werden nicht verknuepft und nicht offengelegt.
- Management-Review-Pakete enthalten einen unveraenderlichen Snapshot. Spaetere Linkaenderungen schreiben alte Pakete nicht um.

Fachlicher Nutzen:

- AI-Systeme werden nicht nur als Produktmerkmal, sondern als steuerbares Risiko- und Governance-Objekt sichtbar.
- AI-Act-, ISMS-, Product-Security- und Evidence-Arbeit laufen ueber dasselbe Nachweis- und Review-Modell.
- Fachliche Reviews koennen frueh erkennen, ob Einstufung, Oversight, Monitoring oder Evidence fehlen.

Die technischen API-, Tenant- und Snapshot-Regeln sind in docs/AIGOVERNANCE.md gebuendelt.

Fuer Nicht-Sicherheitsleute:
Dieser Bereich beantwortet: Welche KI wird genutzt, wofuer, mit welchen Risiken, wer kontrolliert sie und wo ist der Nachweis?

5.17 Supplier Risk:

Zweck:
Externe Abhaengigkeiten als eigenen Risikobereich steuern.

Supplier Risk umfasst Lieferanten, Cloud-Provider, SaaS-Dienste, IKT-Drittdienstleister, Produktzulieferer, Komponentenhersteller und Audit-/Nachweisportale. ISCY fuehrt diese Objekte nicht nur als Adressliste, sondern als Third-Party-Risk-Register.

Aktueller Rust-Funktionsumfang:

- Webansicht /suppliers/ mit Supplier-Risk Register, Score, Kritikalitaet, Review-Status, Evidence-Stand und Exposure
- Web-Detailansicht /suppliers/{id}/ mit Reviewhistorie, Unterauftragnehmern, Vertrags-/Exit-Daten, Evidence-/Control-/Risk-Links und Auditspur
- API GET und POST /api/v1/suppliers fuer Uebersicht und Anlage sowie GET und PATCH /api/v1/suppliers/{id} fuer Detaildaten und Aktualisierung
- API GET und POST /api/v1/suppliers/{id}/reviews fuer Review-/Approval-Ereignisse
- API GET und POST /api/v1/suppliers/{id}/subprocessors sowie PATCH /api/v1/suppliers/{id}/subprocessors/{subprocessorid} fuer Unterauftragnehmer im

Supplier-Kontext
- API GET /api/v1/suppliers/{id}/evidence, POST /api/v1/suppliers/{id}/evidence-links, POST /api/v1/suppliers/{id}/control-links und POST /api/v1/suppliers/{id}/risk-links fuer explizite Nachweis-, Control- und Risiko-Bezuege
- Webansicht /suppliers/product-security/ fuer Supplier/Product Security mit lokaler Advisory-/PSIRT-/CVE-Erfassung, Review-Status, Evidence, Vertragsstatus und Exit-Plan-Status
- API GET und POST /api/v1/suppliers/product-security, GET und PATCH /api/v1/suppliers/product-security/{recordid}, POST /api/v1/suppliers/product-security/{recordid}/status, POST /api/v1/suppliers/product-security/{recordid}/evidence, GET /api/v1/suppliers/product-security/{recordid}/events, GET /api/v1/suppliers/{supplierid}/product-security und GET /api/v1/suppliers/{supplierid}/contract-exit-history
- Datenfelder fuer Vertrags-/Security-Annex-Bezug, Security-Kontakt, Datenarten, Regionen, Exit-Abhaengigkeit, regulatorischen Scope, Review-Status, Review-Faelligkeit und Notes
- additive Migration 0031rustsupplierreviewworkflow fuer Review-Freigaben, Subprocessors, Vertragslaufzeiten, Exit-Test-Nachweise und Link-/Audit-Tabellen
- additive Migration 0034rustsupplierproductsecuritygovernance fuer Supplier/Product-Security-Datensaetze, Evidence-Links, Ereignisse und Vertrags-/Exit-Plan-Historie
- automatische Signale aus Produktkomponenten, offenen Product-Security-Schwachstellen, Supplier-bezogenen Risiken und Supplier-Evidence
- direkte Evidence-Vorbefuellung je Supplier mit stabilem Linked Requirement SUPPLIER:{id}
- Score- und Issue-Logik fuer kritische CVEs, ueberfaellige Reviews, fehlende Evidence, fehlende Exit-Strategie, fehlenden Security-Kontakt und fehlende Risikodokumentation
- Review-Statusmodell: draft, inreview, approved, approvedwithconditions, rejected, expired und archived
- Supplier/Product-Security-Review-Statusmodell: draft, needsreview, inreview, acceptedrisk, remediationrequired, mitigated, closed und notapplicable
- Exit-Test-Statusmodell: notrequired, required, planned, passed, failed und overdue
- Vertrags-/Exit-Plan-Historie mit Version, Akteurreferenz, Aenderungszeitpunkt, Grund, vorherigem/neuem Status, Summary und Evidence-Referenzen
- Begruendungspflicht fuer approvedwithconditions und rejected
- tenantgebundene Validierung fuer Supplier, Subprocessors, Owner, Evidence, Controls und Risiken
- tenantgebundene Filter fuer Lieferant, Produkt/Service, Review-Status, Schweregrad, offene Massnahmen, ueberfaellige Reviews sowie DORA-, NIS2-, DSGVO- und kritische-Service-Relevanz
- auditierbare Erstellung, Aenderung, Statusentscheidung, Subprocessor-Aenderung, Product-Security-Statuswechsel, Evidence-Verknuepfung, Vertrags-/Exit-Plan-Aenderung und Link-Verwaltung ohne Secrets, SQL-Details oder vertrauliche Payloads

Fachlicher Nutzen:

- DORA-IKT-Drittparteienrisiko, NIS2-Supply-Chain-Anforderungen, CRA-Komponenten-/Herstellerbezug, DSGVO-Datenverarbeitung und TISAX-Lieferkettennachweise koennen gemeinsam betrachtet werden.
- Kritische externe Abhaengigkeiten werden sichtbar, bevor sie erst in einem Incident auffallen.
- Evidence, Risiken, Product Security und Roadmap-Arbeit bekommen einen gemeinsamen Lieferantenbezug.
- Review-Entscheidungen werden nicht nur als aktueller Status gespeichert, sondern als nachvollziehbare Freigabehistorie.
- Vertragsende, automatischer Renew, Kuendigungsfrist und Exit-Test-Status helfen dabei, Abhaengigkeiten vor Ablauf, Eskalation oder Lieferantenwechsel zu steuern.
- Lokale Advisory-/PSIRT-/CVE-Metadaten machen sichtbar, welche Lieferantenprodukte, Versionen, Nachweise und Massnahmen fuer NIS2, DORA, DSGVO, CRA und Auditvorbereitung relevant sind.

Rollen- und Sicherheitsmodell:

- Admin- und Editor-Rollen duerfen Supplier, Reviews, Subprocessors, Supplier/Product-Security-Datensaetze, Status, Evidence-Links und Links schreiben.
- Read-only-Rollen sehen sichere tenantgebundene Metadaten.
- Fremde Tenant-Objekte, manipulierte Supplier-IDs sowie fremde Evidence-, Risk- und Owner-IDs werden nicht aufgeloest.
- Advisory-Referenzen werden nur als lokale Text-/URL-Metadaten gespeichert; ISCY ruft keine externen Feeds automatisch ab und rendert unsichere URL-Schemes nicht als Link.
- Interne Store- oder SQL-Fehler werden in API und Web UI nicht als technische Details ausgegeben.

Bewusst nicht Teil dieses Moduls:

- keine neue Evidence-Engine
- keine neue Risiko-Engine
- keine neue Control-Bibliothek
- keine eigene freie NVD-, Hersteller-, GitHub-Advisory- oder sonstige Live-Feed-Integration innerhalb des Supplier-Moduls; offizielle NVD-, CISA-KEV- und FIRST-EPSS-Daten werden ausschliesslich ueber den zentralen, fest erlaubten Dienst aus 5.18 synchronisiert
- keine eigenen Supplier-spezifischen Legal-Hold-, Disposition-, Loesch-, Re-Hash- oder Objektspeicher-Engines

Fuer Nicht-Sicherheitsleute:
Dieser Bereich beantwortet: Von welchen externen Parteien haengt unser Betrieb ab, wie kritisch sind sie, welche Nachweise fehlen und wo entsteht daraus Risiko?

5.18 Vulnerability Intelligence:

Zweck:
Bekannte Schwachstellen fachlich und technisch bewerten.

Aktueller LLM-Status:
Der CVE-Bereich besitzt einen optionalen runllm-Pfad, die Route POST /api/v1/llm/generate und die Webansicht /cves/llm-test/. Diese Funktionen sind im aktuellen Rust-Stand ein deterministischer LLM-Stub. Sie erzeugen strukturierte technische und Management-Zusammenfassungen, Massnahmenvorschlaege und Evidence-Hinweise direkt im Rust-Code; dabei wird kein Qwen-, llama.cpp-, Ollama- oder anderes Sprachmodell aufgerufen.
Neue Ausgaben besitzen die feste Kennung iscy-rust-llm-stub-v1, Backend ruststub und generationmethod=deterministicstub. Die Herkunft bestaetigt ausdruecklich inferenceperformed=false, modelconfigurationapplied=false und requireshumanreview=true.
Neue CVE-Assessment-Ausgaben speichern STUBGENERATED und werden als Regelbasiert (Stub) angezeigt. Historische Ausgaben werden nicht nachtraeglich umetikettiert; runllm=false erhaelt bestehende Texte und Herkunftsmetadaten. Der Kompatibilitaetszaehler llmgenerated umfasst beide generierten Assistenzformen und ist kein Inferenznachweis.

LOCALLLMMODELNAME, Modellpfad, Kontext, Threads und beide bisherigen GPU-Layer-Variablen werden vom Stub ignoriert. Die Runtime-Anzeige meldet keinen Modellimport, keine bereite Modellruntime und keine angewendeten Inferenzparameter. Eine RAG-/Embedding-/Retrieval-Runtime ist nicht Bestandteil des aktuellen ISCY-Backends.

Was der Bereich jetzt leisten soll:

- einzelne CVEs und begrenzte UTC-Deltas aus der offiziellen NVD API 2.0 laden
- CVE-Referenzdaten global, idempotent und mit Checkpoint, gezaeuntem Lease und Laufstatus pflegen
- den offiziellen CISA-KEV-Katalog atomar an vorhandene CVEs anbinden
- FIRST-EPSS-Score, Percentile und Modelldatum in begrenzten, auch bei fehlenden Scores fair fortschreitenden Batches pflegen
- vorhandene tenantgebundene Assets, Softwarekomponenten und SBOMs ueber nachvollziehbare CPE-/Versionsregeln korrelieren
- unsichere Matches als Pruefkandidaten statt als bestaetigte Betroffenheit behandeln
- vorhandene Product-Security-Vulnerability-Findings idempotent aktualisieren
- CVSS, KEV, EPSS, Assetkritikalitaet, Datenalter, VEX und dokumentierte Compensating Controls transparent priorisieren
- ausschliesslich passive Pruef-, Patch- und Mitigationsempfehlungen ausgeben

NVD, CISA KEV und FIRST EPSS sind feste offizielle HTTPS-Quellen. Freie URLs, Redirects, lokale Dateien sowie private oder Link-Local-Ziele werden nicht akzeptiert. ISCY bleibt ohne Feed-Verbindung startfaehig; vorhandene Daten und Findings bleiben bei einem Ausfall erhalten. Ein optionaler NVD-Key wird ueber die vorhandene NVDAPIKEYFILE-Secret-Abstraktion gelesen und weder protokolliert noch gespeichert.

Ein KEV-Treffer bedeutet bekannte Ausnutzung der Schwachstelle, aber keine Kompromittierung des eigenen Assets. EPSS ist eine Wahrscheinlichkeitsschaetzung und kein Angriffsnachweis. Ein CVE- oder Softwarematch bleibt Vulnerability Finding beziehungsweise Review-Kandidat, nicht Incident. Sync, Korrelation und Triage erzeugen weder Evidence noch Agentenbefehle oder aktive Reaktionen. Ohne belastbare Quelle bleibt EOL/EOS

UNKNOWN.

Ein vorhandenes Product-Security-Vulnerability-Finding wird nur mit bereits vorhandenem tenantgebundenem Produktbezug erzeugt oder aktualisiert. Reine Asset-Matches bleiben als erklaerbare Korrelationen im bestehenden Review; ISCY erzeugt dafuer kein kuenstliches Produkt und kein paralleles Finding-Modell. Komplexe NVD-CPE-Kontexte mit logischem AND oder nicht-vulnerablen Plattformbedingungen werden nicht zu einem bestaetigten Match vereinfacht, sondern bleiben pruefpflichtig. Ein aelterer oder in der Datensatzanzahl inkonsistenter KEV-Katalog wird fail-closed abgewiesen.

Tenantrollen sehen den fachlichen Feedstatus, aber keine globalen Checkpoint-/Fenster-Zeitpunkte oder anfordernden Plattform-Actor-IDs. Diese Betriebsdaten sind auf die explizite globale Sync-Permission und Superuser begrenzt.

Fachlicher Nutzen:

- bessere Priorisierung als nur CVSS
- Verbindung zwischen Technik, Betrieb und Regulierung
- Grundlage fuer PSIRT, Patch-Steuerung und Management-Kommunikation

Fuer Nicht-Sicherheitsleute:
Hier wird aus einer technischen Schwachstellenmeldung eine geschaeftlich nutzbare Bewertung.

Technische Details, Rollen und Betriebsgrenzen stehen in docs/VULNERABILITYINTELLIGENCESOFTWAREHYGIENE.md; die verbindliche Architekturentscheidung steht in docs/ADRCONTINUOUSVULNERABILITYINTELLIGENCESOFTWAREHYGIENE.md.

5.19 Native Threat Intelligence und Security Observations:

Zweck:
Begrenzte Threat Indicators und bereits vorhandene technische Findings in einer tenantgebundenen Analystenansicht gemeinsam triagieren.

Phase 1 unterstuetzt lokal validierte IPv4-, IPv6-, Domain-, HTTP(S)-URL- und SHA-256-Indicators. Jeder Indicator besitzt Source-/Provenance-Angaben, Confidence, Gueltigkeit, Lifecycle und Klassifizierung. Die Validierung erfolgt ohne DNS-, HTTP-, Reputation- oder Feed-Aufruf.

Security Observations koennen manuell erfasst werden oder auf ein bestehendes Agent Finding beziehungsweise Product-Security-Vulnerability-Finding zeigen. Diese vorhandenen Datensaetze bleiben die kanonische Quelle. Die Observation ist nur eine begrenzte Triage-Projektion mit stabiler Source-Referenz, Kategorie, Severity, Owner und Status.

Indicators werden in Phase 1 ausschliesslich manuell mit Observations verknuepft. Match-Typ, Begruendung, Evaluator und Triage werden auditierbar festgehalten. Daraus entsteht weder automatisch ein Incident noch ein Evidence-Datensatz, Risiko, Roadmap-Task oder eine technische Reaktion.

Die Rollen sind bewusst getrennt:

- SOCANALYST liest Indicators und Observations und darf Observations sowie manuelle Links triagieren.
- SECURITYADMIN darf zusaetzlich Indicators verwalten und Observations erfassen.
- bestehende Rollen erhalten keine automatische neue Permission-Zuweisung.

Die API liegt unter /api/v1/threat-intelligence/ und /api/v1/security-observations/; die Webarbeitsansicht liegt unter /security-observations/. Weitere Betriebs- und Sicherheitsgrenzen stehen in docs/THREATINTELLIGENCEOBSERVATIONS.md und der zugehoerigen ADR.

Fuer Nicht-Sicherheitsleute:
Der Bereich hilft, einen Hinweis nachvollziehbar zu bewerten, ohne ihn automatisch zu einem meldepflichtigen Vorfall oder zu einer technischen Gegenmassnahme zu machen.

5.20 Software Approval und Exception Policy:

Zweck:
Fuer ein konkretes vorhandenes Softwareobjekt nachvollziehbar festhalten, ob es freigegeben, eingeschraenkt oder untersagt ist und ob eine befristete Ausnahme wirksam ist.

Policies koennen exakt auf ein tenantgebundenes Produkt, Asset, eine kanonische Produktkomponente oder eine bereits importierte SBOM-Komponente zeigen. Freier Text, unscharfe Herstellernamen, Wildcards, Regex, Versionsausdruecke und neue PURL-/CPE-Heuristiken sind keine technische Match-Grundlage.

Die effektiven Zustaende bedeuten:

- APPROVED: Eine aktuell wirksame Policy gibt das exakte Ziel ausdruecklich frei und keine staerkere Restriktion gilt.
- RESTRICTED: Mindestens eine einschraenkende Policy bleibt wirksam.
- PROHIBITED: Mindestens eine untersagende Policy bleibt wirksam.
- EXCEPTIONACTIVE: Jede wirksame restriktive Policy ist durch ihre eigene genehmigte und noch gueltige Ausnahme abgedeckt. Das ist keine normale Freigabe.
- UNMANAGED: Fuer das Ziel gibt es keine aktuell wirksame Policy.
- REVIEWREQUIRED: Die Bewertung ist unvollstaendig oder inkonsistent und darf nicht als Freigabe verstanden werden.

Ohne wirksame Ausnahme gewinnt immer die restriktivste passende Entscheidung: PROHIBITED vor RESTRICTED vor APPROVED. Eine Ausnahme wirkt ausschliesslich fuer ihre referenzierte Policy und ihr exaktes Ziel. Sie veraendert weder die Policy noch CVE-, VEX-, Risk-Acceptance- oder manuelle Triage-Daten.

Ausnahmen durchlaufen DRAFT, PENDINGREVIEW und danach APPROVED oder REJECTED; genehmigte Ausnahmen koennen REVOKED oder EXPIRED werden. Jede Ausnahme braucht ein zukuenftiges Ablaufdatum. Der Server prueft Start und Ablauf in UTC bei jeder aktuellen Bewertung. Deshalb bleibt eine abgelaufene Ausnahme auch ohne Scheduler unwirksam. Antragsteller duerfen ihre eigene Ausnahme niemals genehmigen oder ablehnen.

Rollen:

- SECURITYADMIN und COMPLIANCEMANAGER: tenantlokale Gesamtbearbeitung
- SOCANALYST: lesen, passiv bewerten und Ausnahme beantragen
- AUDITOR: Policies und Audit lesen
- Admin, Staff und Superuser: bestehende administrative Semantik
- direkte und gruppenbasierte Permissions bleiben moeglich; Self-Approval ist trotzdem gesperrt

Die Webansicht liegt unter /software-policies/. Die API umfasst /api/v1/software-policies und /api/v1/software-policy-exceptions. Revisionen verhindern verlorene Updates; PostgreSQL-Locks, SQLite-Schreibserialisierung, Tenant-Foreign-Keys und atomare Audittransaktionen sichern parallele Entscheidungen.

Die Auswertung bleibt passiv. Sie installiert, blockiert, deinstalliert oder isoliert keine Software, sendet keinen Agentenbefehl und erzeugt weder Incident, Evidence, Security Observation, VEX noch Risk Acceptance.

Keine Policy bedeutet nicht freigegeben. Keine bekannte Schwachstelle bedeutet ebenfalls nicht freigegeben. Eine Policy-Freigabe ist kein Nachweis, dass Software frei von Schwachstellen ist.

Technische Details stehen in docs/SOFTWAREAPPROVALEXCEPTIONPOLICY.md; die verbindliche Praezedenz- und Architekturentscheidung steht in docs/ADRSOFTWAREAPPROVALEXCEPTIONPOLICY.md.

6. Typische Arbeitsablaeufe:

6.1 Erstaufbau eines ISMS:

Der vorgesehene Ablauf bleibt bestehen. Der Wizard-Schritt ist wegen der in 5.2 beschriebenen Migrationsluecke derzeit blockiert und nicht durch einen externen Ersatzprozess abgenommen. Ein eigenstaendiger SoA-Erstentwurf und die

Control-Bearbeitung im Status DRAFT sind verfuegbar. Der in 5.10 abgegrenzte Nachfolgeversions-/Freigabeablauf fuer gesperrte SoA-Dokumente bleibt offen.

1. Tenant anlegen beziehungsweise Organisationsprofil pflegen
2. Wizard durchlaufen
3. Prozesse, Assets und Lieferanten importieren
4. Requirements und Applicability sichten
5. erste Risiken erfassen
6. Evidence aufbauen
7. SoA generieren und fachlich pruefen
8. Report und Roadmap erzeugen

6.2 Auditvorbereitung:

1. offene Findings und Reviews sichten
2. Evidence Needs synchronisieren
3. Nachweise aktualisieren
4. SoA fachlich pruefen; Erstentwuerfe und Control-Bearbeitung sind im Status DRAFT verfuegbar. Fuer gesperrte Dokumente bleibt der Nachfolgeversions- und Freigabeablauf offen (siehe 5.10).
5. Audit-Report erzeugen

6.3 NIS2-orientierte Arbeitsweise:

1. Tenant als NIS2-relevant markieren
2. kritische Services klar beschreiben
3. Prozesse, Assets und Risiken auf wesentliche Dienste beziehen
4. Schwachstellen mit NIS2-Kontext bewerten
5. Reports fuer Leitung und Nachweiszwecke verwenden

6.4 Produkt- und Schwachstellensteuerung:

1. Produkte, Releases und Komponenten pflegen
2. CSAF-Advisories oder SBOMs importieren und Validierungsfehler pruefen
3. SBOM-Importstaende vergleichen und neue, entfernte oder geaenderte Komponenten bewerten
4. VEX-Status fuer Schwachstellen dokumentieren: betroffen, nicht betroffen, behoben oder in Untersuchung
5. Komponenten-Matches ueber CPE oder PURL kontrollieren
6. CVE-Asset-Korrelationen vorschlagen lassen
7. fuer exakte Produkte, Assets oder Komponenten Software-Policies pflegen
8. UNMANAGED und REVIEWREQUIRED fachlich pruefen
9. befristete Ausnahmen getrennt beantragen und unabhaengig entscheiden
10. Korrelationen fachlich akzeptieren oder ablehnen
11. Aus akzeptierten Korrelationen Risiko- und Roadmap-Arbeit erzeugen
12. CVE-Risiko-Review-Queue abarbeiten
13. Evidence direkt aus Queue, Risiko oder Roadmap-Task hochladen
14. CRA-Readiness je Produkt pruefen und Massnahmen ueber Roadmap oder Risiko-Behandlung steuern
15. Supplier/Product-Security-Datensaetze fuer relevante Lieferantenprodukte pflegen, Advisory-/PSIRT-/CVE-Bezuege lokal dokumentieren und Evidence verknuepfen
16. Vertrags-/Exit-Plan-Status fuer kritische Lieferantenprodukte pruefen und Review-Pakete fuer NIS2, DORA oder DSGVO vorbereiten

6.5 Incident- und NIS2-Meldeworkflow:

Die Rust-Webroute /incidents/ fuehrt operative Sicherheitsvorfaelle als mandantenfaehige Fallakten. Ein Incident kann Reporter, Owner, Risiko, Asset und Prozess referenzieren und enthaelt Typ, Runbook, Status, Severity, Stakeholder-Zusammenfassung sowie Behoerden- oder Case-Referenz. Tenantbezogene Runbook-Vorlagen werden in incidentsrunbooktemplate gepflegt, per /api/v1/incidents/runbook-templates ausgeliefert und koennen beim Anlegen eines Incidents direkt als bearbeitbare Startvorlage uebernommen werden. Runbook-Inhalte sind auf 64 KiB und 100 materialisierbare Schritte begrenzt; groessere neue oder vorhandene Werte werden vor Datenbank-Inserts abgewiesen.

ISCY trennt bewusst zwischen einem Security Incident und einem erheblichen Sicherheitsvorfall. Nicht jeder operative Sicherheitsvorfall ist automatisch NIS2-meldepflichtig. Die Fallakte kann deshalb zuerst als normaler Incident entstehen und danach fachlich bewertet werden.

Die NIS2-Erheblichkeitsentscheidung wird in ISCY mit vier Status gefuehrt:

1. Nicht bewertet
2. Nicht erheblich

3. Wahrscheinlich erheblich
4. Erheblich / NIS2 meldepflichtig

Zur Begruendung koennen Kriterien, Entscheidungstext, Referenz und Bewertungszeitpunkt gepflegt werden. Fachlich orientiert sich die Bewertung an NIS2 Art. 23. Die Durchfuehrungsverordnung (EU) 2024/2690, insbesondere Art. 3, wird in ISCY als Best-Practice-Referenz genutzt, auch wenn ihr unmittelbarer Pflichtbereich nur bestimmte Entitaeten betrifft. Dadurch wird sichtbar, warum ein Fall meldepflichtig ist oder warum er bewusst nicht als erheblicher Sicherheitsvorfall behandelt wird.

Erst wenn ein Incident auf Erheblich / NIS2 meldepflichtig gesetzt wird, berechnet ISCY die relevanten Fristen aus dem Erkennungszeitpunkt:

1. 24h-Fruehwarnung
2. 72h-Meldung
3. Abschlussbericht nach 30 Tagen

Die Uebersicht zeigt offene Faelle, Erheblichkeitsstatus, NIS2-relevante Faelle und ueberfaellige Meldeschritte. Gesendete Meldungen koennen ueber die API als Zeitstempel gepflegt werden. Damit vermeidet ISCY eine Angstlogik nach dem Motto "24 Stunden fuer alles" und erzwingt stattdessen eine nachvollziehbare fachliche Entscheidung.

Die Detailseite /incidents/{id} dient als operative Fallakte. Dort koennen berechtigte Rollen Typ, Runbook, Status, Severity, Erheblichkeitsentscheidung, Behoerdenreferenz, Zeitlinie und Meldezeitpunkte pflegen; die Runbook-Bibliothek wird als Referenz direkt neben der Fallakte angezeigt. Eine Entscheidungsleiste fuehrt von Vorfall ueber Erheblichkeit und Bearbeitung bis zum Meldepaket und springt direkt in die passenden Abschnitte. ISCY dokumentiert die Anlage der Fallakte, Statuswechsel, NIS2-Erheblichkeitsaenderungen, automatisch angeforderte Reviews, manuelle Timeline-Notizen und incidentbezogene Evidence-Uploads als Timeline-/Audit-Events mit Actor, Zeitpunkt, Ereignisart und Detailtext. Ein Incident wird dabei nicht automatisch als erheblicher Sicherheitsvorfall behandelt: Wird eine Fallakte als Nicht erheblich bewertet, fordert ISCY automatisch Review/Freigabe an und laesst die NIS2-Meldefristen inaktiv, bis die Einstufung ggf. auf Erheblich / NIS2 meldepflichtig geaendert wird. Manuelle Notizen koennen auch ueber POST /api/v1/incidents/{id}/timeline-notes automatisiert erfasst werden. Evidence-Uploads koennen ueber incidentid direkt an einen Incident gekoppelt werden und erscheinen in der Fallakte; fuer berechtigte Rollen steht der Upload direkt auf der Incident-Detailseite bereit. Alertmanager-Fallakten werden ueber Fingerprint oder Alertname dedupliziert; resolved Alerts schliessen passende offene Fallakten automatisch und sind in /operations/incidents/ als Betriebsuebersicht mit Filtern fuer open, critical und resolved sichtbar. Optional kann ISCYALERTMANAGERREQUIRERESOLUTIONREVIEW=1 gesetzt werden, damit automatisch geschlossene Alert-Fallakten ohne Lessons Learned als Review-Pflicht fuer Root Cause und Lessons Learned markiert werden. Das Meldepaket unter /incidents/{id}/nis2-export buendelt Fallakte, NIS2-Erheblichkeitsentscheidung, regulatorische NIS2/DORA/DSGVO-Entscheidungsmatrix, Runbook, verknuepfte Evidence, Audit-Timeline, betroffene Bezuege, 24h-/72h-/30-Tage-Fristen, Stakeholder-Zusammenfassung und Lessons Learned; zusaetzlich stehen HTML und PDF ueber /incidents/{id}/nis2-export.html und /incidents/{id}/nis2-export.pdf bereit. DORA-Pruefpakete koennen ueber /incidents/{id}/dora-export, /incidents/{id}/dora-export.html und /incidents/{id}/dora-export.pdf erzeugt werden; DSGVO-Pruefpakete entsprechend ueber /incidents/{id}/dsgvo-export, /incidents/{id}/dsgvo-export.html und /incidents/{id}/dsgvo-export.pdf. Das Dashboard zeigt zudem Incidents ohne abgeschlossene Erheblichkeitsbewertung als klickbare Kennzahl und fuehrt direkt zur gefilterten Incident-Liste.

6.6 SOC-Playbook fuer Phishing- und aehnliche Incident-Faelle:

Empfohlene Kette fuer die operative Bearbeitung:

1. Scope bestimmen
Klaeren, wer und was betroffen ist, seit wann der Vorfall laeuft und welche Systeme, Konten, Daten oder Geschaeftsbereiche im Risiko stehen.
2. Informationen korrelieren
Mail-Logs, SEG, SIEM, EDR, Auth-, Proxy-, DNS- und Firewall-Daten zusammenfuehren.
3. Nach Gemeinsamkeiten suchen
IOCs und TTPs vergleichen (Domain, URL, Hash, Prozesse, Zeitfenster, Zielgruppe).
4. Vorfall bewerten
Einordnen, ob Spam, Phishing, BEC, Malware Delivery oder bereits Account Compromise vorliegt.
5. Verdacht bestaetigen
Von der Hypothese in die Incident-Response wechseln, sobald belastbare Evidenz vorliegt.
6. Priorisieren
Kritikalitaet und Dringlichkeit anhand Impact, Privilege-Level, Ausbreitungs- und Datenabflussrisiko einstufen.

7. Dokumentieren
Nachvollziehbar festhalten, was bekannt ist, was vermutet wird, was getan wurde und warum.
8. Containment einleiten
Sofortmassnahmen risikobasiert umsetzen (z. B. Mail entfernen, URL blockieren, Session widerrufen, Konto sichern, Host isolieren).
9. Gegebenenfalls eskalieren
An L2/L3, IR, IAM, Management, Datenschutz oder Legal uebergeben, wenn Risiko, Scope oder Komplexitaet es erfordern.

Merksatz: Erst verstehen, dann bewerten, dann eindaemmen, dann eskalieren - wenn Risiko oder Komplexitaet es verlangen.

6.7 ISCY lokal auf NixOS starten:

Einfachster lokaler Start:

```
cd ~/Projekte/ISCY
./start.sh
```

Danach ist ISCY unter http://127.0.0.1:9000/login/ erreichbar.

Demo-Login:

admin / Admin123!

Ohne Wrapper kann der Rust-Service so initialisiert und gestartet werden:

```
nix run .#iscy-backend -- init-demo
DATABASE_URL=sqlite:///db.sqlite3 RUST_BACKEND_BIND=127.0.0.1:9000 nix run .#iscy-backend
```

Produktive Erstinitialisierung erfolgt ohne Demo-Seed. Dafuer wird ein eigenes Passwort aus einer Secret-Datei gelesen und ein initialer Admin angelegt:

```
ISCY_INITIAL_ADMIN_PASSWORD_FILE=/run/secrets/iscy_initial_admin_password \
nix run .#iscy-backend -- init-admin
```

Fuer Production unterstuetzt ISCY file-basierte Quellen fuer Datenbank, NVD, initialen Admin, Alertmanager und S3-kompatiblen Evidence Storage. Ein direkter Wert und seine FILE-Quelle duerfen nie gleichzeitig gesetzt sein. Die Dateien muessen regulaer, symlinkfrei, hoechstens 16 KiB gross und ohne Group-/Other-Rechte sein. Standardmaessig werden nur Dateien unter /run/secrets akzeptiert; Production Compose bindet das konfigurierte Secret-Verzeichnis dort read-only ein. Fehlermeldungen nennen keine Secretwerte oder credential-haltigen URLs. Vollstaendige Regeln und Rotationshinweise stehen in SECURITY.md und docs/CONFIGURATION.md.

Maschinenlesbarer Betriebsstatus fuer lokale Pruefung, Monitoring und Agenten:

```
curl -fsS http://127.0.0.1:9000/health/live
curl -fsS http://127.0.0.1:9000/status/operations.json
curl -fsS http://127.0.0.1:9000/metrics
```

Alertmanager kann Betriebsalarme an ISCY melden:

```
curl -fsS -X POST http://127.0.0.1:9000/api/v1/operations/alertmanager \
-H 'content-type: application/json' \
-d '{"receiver":"iscy-operations","status":"firing","alerts":[]}'
```

Ohne Tenant-/User-Kontext wird der Alert nur normalisiert. Mit schreibendem Tenant-Kontext erzeugt ISCY fuer firing Alerts automatisch eine Incident-Fallakte, verknuepfte Evidence und einen Timeline-Eintrag. Wiederholte firing Alerts werden dedupliziert, resolved Alerts schliessen die passende offene Alert-Fallakte automatisch. Die Alert-Operations-Seite /operations/incidents/ bietet direkte Filter fuer open, critical und resolved. Wird ISCYALERTMANAGERREQUIRERESOLUTIONREVIEW=1 gesetzt, markiert ISCY automatisch geschlossene Alert-Fallakten ohne Lessons Learned als Review-Pflicht. Das Monitoring-Beispiel nutzt fuer lokale Demo-Stacks Tenant 1, User 2 und Rolle CONTRIBUTOR; User 2 ist der per Demo-Seed angelegte technische Operations-User ops-alertmanager.

Mit Tenant-Kontext enthaelt der Betriebsstatus zusaetzlich fachliche Signale zu ISCY-27, Supplier-Risk, Product Security, AI Governance, offenen CVE-Reviews, fehlender Evidence, Migrationen, Runtime-Flags und verbundenen Rust-Modulen:

curl -fsS -H 'x-iscy-tenant-id: 1' -H 'x-iscy-user-id: 1' \
'http://127.0.0.1:9000/api/v1/status/operations?tenant_id=1&user_id=1'
curl -fsS -H 'x-iscy-tenant-id: 1' -H 'x-iscy-user-id: 1' \
'http://127.0.0.1:9000/api/v1/status/metrics?tenant_id=1&user_id=1'

Die Prometheus-/Grafana-Betriebsdoku liegt in docs/OPERATIONSMONITORING.md.

Fuer Community-Readiness und Production-Hardening sind zusaetzlich diese Dokumente verbindlich:

- docs/COMMUNITYREADINESSPHASE0PHASE1.md beschreibt die aktuelle Gap-Liste und den Status READY WITH DOCUMENTED LIMITATIONS.
- docs/CONFIGURATION.md dokumentiert Betriebsmodi, Secrets, Proxy-Grenzen und sichere Defaults.
- docs/TLSANDREVERSEPROXY.md beschreibt HTTPS-Terminierung, HSTS und die Behandlung von x-iscy- Identity-Headern.
- docs/AUTHORIZATIONMODEL.md beschreibt Session-, Header- und Rollenmodell inklusive Negativtests.
- docs/PRODUCTIONHARDENING.md beschreibt den Production-Preflight, Security-Header und offene Phase-1-Risiken.

Im Production-Modus (ISCYAPPMODE=production) bricht ISCY den Start ab, wenn kritische Annahmen fehlen: keine Datenbank, Beispiel-Secrets, aktive Demo-Zugangsdaten, Demo-Seeding, unsichere Cookies, oeffentliche Bind-Adresse ohne bestaetigten Reverse Proxy, HSTS ohne bestaetigtes HTTPS oder fehlendes Alertmanager-Secret. Normale Clients duerfen x-iscy-tenant-id, x-iscy-user-id oder x-iscy-roles produktiv nicht zur Identitaetssteuerung verwenden; diese Header werden nur akzeptiert, wenn ein vertrauenswuerdiger Proxy das explizit absichert.

Lokale Logins sind gegen einfache Brute-Force-Schleifen begrenzt: Nach fuenf fehlgeschlagenen Versuchen pro Tenant/Username blockiert ISCY weitere Versuche fuer 15 Minuten. Mit angewendeter Migration 0023rustsecurityruntimestate wird dieser Zustand im Security-Store persistiert und kann von mehreren Backend-Instanzen gemeinsam genutzt werden. Der Alertmanager-Webhook kann zusaetzlich zum Bearer-Token mit ISCYALERTMANAGERHMACSECRETFILE signierte timestamp.body-Requests verlangen; das Replay-Fenster wird ueber ISCYALERTMANAGERHMACMAXAGESECONDS gesteuert. Optional kann x-iscy-alert-nonce gesendet werden; bei aktivem Security-Store speichert ISCY verwendete Nonces im Replay-Fenster und lehnt Wiederholungen ab.

Die Tenant-Isolation wird nicht nur auf Listen, sondern auch auf sensible Objektoperationen angewendet. Negativtests decken fremde Detail-, Write- und Exportzugriffe fuer Kernobjekte ab, darunter Incidents mit NIS2-/DORA-/DSGVO- und Timeline-Exporten sowie Management-Review-Pakete. Evidence-Uploads duerfen keine Session und keinen Incident eines fremden Tenants referenzieren. ISCY antwortet in diesem Fall ohne Fremdmandantendaten mit 400 invalidevidenceupload und entfernt die bereits temporaer gespeicherte Datei.

Fuer den direkten Monitoring-Betrieb liegen diese Artefakte im Repository:

- deploy/monitoring/prometheus/iscy-scrape.yml
- deploy/monitoring/prometheus/iscy-operations-alerts.yml
- deploy/monitoring/alertmanager/iscy-alertmanager.yml
- deploy/monitoring/grafana/iscy-operations-dashboard.json
- deploy/monitoring/docker-compose.yml
- deploy/monitoring/nixos/iscy-monitoring.nix
- deploy/monitoring/nixos/example-host.nix

Die Statusseite /status/ zeigt neben Health, Migrationen, Modulen, offenen Signalen und Prometheus-Scrape-Konfiguration auch einen kompakten Grafana-Query-Spickzettel sowie direkte Links zu Incident-Fallakten und /operations/incidents/. Das Grafana-Dashboard enthaelt zusaetzlich Panels fuer Alert-Incidents mit konfigurierbarer iscybaseurl, konkretem Incident-Drilldown ueber iscyoperationsalertmanagerincidentinfo, Product-Security-Coverage, CVE-Review-Trend und Importvalidierung.

Der Product-Security-Bereich zeigt zusaetzlich Trenddaten fuer SBOM-/CSAF-/Threat-Coverage, offene CVE-Reviews, fehlende Evidence, Importvalidierung, Snapshot-Verlauf, CRA-Readiness und SBOM-Diffs. Maschinenlesbar sind diese Daten ueber GET /api/v1/product-security/trends, GET /api/v1/product-security/products/{productid}/cra-readiness, GET /api/v1/product-security/sbom-diff und ueber Prometheus-Metriken wie iscyproductsecuritytrendsignal, iscyproductsecuritycoveragepercent und iscyproductsecurityimportvalidationtotal.

Runbook fuer automatisch erzeugte Alert-Incidents:

1. Neue Fallakte in /operations/incidents/ sichten, bei Bedarf nach open, critical oder resolved filtern und in /incidents/ oeffnen.
2. Severity, Scope und betroffene Services pruefen.
3. Automatische Evidence kontrollieren und bei Bedarf Grafana-/Log-Nachweise nachreichen.
4. Owner, Eindaemmung, Kommunikation, NIS2-Erheblichkeit und weitere regulatorische Relevanz bewerten.
5. Nach Behebung Timeline, Root Cause, Lessons Learned und Alert-Schwelle reviewen.

Wichtige lokale Pruefbefehle:

```
nix develop --command make rust-smoke
nix develop --command make rust-restore-smoke
ISCY_POSTGRES_RESTORE_DRILL_SOURCE_URL=postgresql://isms:<password>@localhost:5432/iscy_drill_source
\
ISCY_POSTGRES_RESTORE_DRILL_RESTORE_URL=postgresql://isms:<password>@localhost:5432/iscy_drill_restore
\
nix develop --command make rust-postgres-restore-drill
nix develop --command make team-test
make rust-test
make rust-smoke
make team-test
nix flake check
```

rust-restore-smoke erzeugt ueber die laufende Rust-API einen Evidence-Upload, restauriert SQLite-Datenbank und Media-Verzeichnis und prueft danach die DB-Dateireferenz sowie die unveraenderte SHA-256-Pruefsumme. Damit wird nicht nur die Existenz irgendeiner Datei, sondern die Integritaet eines zusammengehoerigen Evidence-Artefakts getestet.

Agent-Payload lokal testen:

```
nix run .#iscy-agent -- --self-test
```

Im Browser kann ein Administrator danach /zero-trust/ oeffnen und Agent hinzufuegen waehlen. Der Assistent erzeugt das Token erst nach der Zusammenfassung und zeigt es genau einmal in einer nicht cachebaren Installationsansicht. Unter /zero-trust/ bleiben anschliessend nur sichere Metadaten, Lifecycle, Policy-Zuordnung, letzter Versuch und Flottenstatus sichtbar.

Agent an eine lokale ISCY-Instanz melden lassen:

```
ISCY_BACKEND_URL=http://127.0.0.1:9000 \
ISCY_TENANT_ID=1 \
ISCY_USER_ID=1 \
nix run .#iscy-agent
```

Produktive Agenten sollten mit Enrollment-Token aufgenommen werden:

```
ISCY_BACKEND_URL=http://127.0.0.1:9000 \
ISCY_TENANT_ID=1 \
ISCY_AGENT_ENROLLMENT_TOKEN=<token> \
nix run .#iscy-agent
```

Danach nutzt der Agent automatisch seinen lokalen State. Fertige Betriebsbeispiele fuer systemd, NixOS, Windows Scheduled Tasks und macOS LaunchDaemons liegen unter deploy/agent/; Details stehen in docs/ZEROTRUSTAGENT.md.

Merksatz: Erst ISCY starten, dann den Agent per --self-test pruefen, einmalig mit Token enrollen und anschliessend ohne Token periodisch betreiben.

6.7 Docker-Betrieb in einfacher Sprache:

Wenn du ISCY schnell und reproduzierbar starten willst, nutze Docker.

1. make docker-check
Prueft zuerst alle Compose-Dateien auf Syntax und Zusammenbau.
2. make docker-smoke
Startet eine kurze Rust-Funktionsprobe (DB, Healthcheck und zentrale API-Probes) und faehrt danach wieder herunter.
3. Danach den gewuenschten Modus starten
- lokal: make dev-up
- stage: make stage-up
- produktiv: make prod-up; make prod-up-llm legt derzeit nur das LLM-Kompatibilitaetsprofil

auf und startet keine echte Modellinferenz

Merksatz: Erst validieren, dann kurz testen, dann dauerhaft starten.

6.8 CRA, IoT/Cloud, Windows, OT und Produktionsarchitektur (einfach erklaert):

Stand dieser Anleitung: 18. April 2026.

CRA (Cyber Resilience Act) in kurz

- Der CRA ist seit Dezember 2024 in Kraft.
- Meldepflichten starten ab 11. September 2026.
- Die wesentlichen Pflichten gelten ab 11. Dezember 2027.

Fuer ISCY bedeutet das praktisch:

1. Produkt-Scope und Komponenten sauber pflegen.
2. Schwachstellen- und Patch-Prozess dokumentieren.
3. Incident- und Meldewege frueh vorbereiten.
4. Nachweise (Evidenz) revisionsfest ablegen.

IoT- und Cloud-Security in kurz

- IoT: Asset-Inventar, sichere Defaults, Updatefaehigkeit, Segmentierung.
- Cloud: Identitaeten haerten, Logs zentral sammeln, Konfigurationen regelmaessig pruefen.
- In ISCY: Prozesse, Assets, Risiken und Massnahmen zusammenfuehren.

Windows-Hardening in kurz

- Admin-Rechte minimieren.
- MFA, starke Authentisierung und Session-Kontrollen aktivieren.
- Security-Baselines und Patch-Stand regelmaessig pruefen.
- Ueber ISCY als Prozess + Evidenz + Review dokumentieren.

OT-Security und Produktionssysteme (OPC UA, MES, SCADA, IdentPro)

- IT und OT klar trennen (Zonen/Segmente).
- Fernzugriffe streng kontrollieren.
- Kritische Schnittstellen (z. B. OPC UA, MES, SCADA, Identitaetsprovider) inventarisieren.
- Notfall- und Wiederanlaufablaeufe fuer Produktion ueben.
- In ISCY als Risiko, Massnahme, Evidenz und Incident-Playbook abbilden.

Merksatz: Technik, Organisation und Nachweis muessen zusammenpassen.

6.9 Einfache Abnahme-Checkliste fuer die neuen Themen:

Nutze diese Liste als schnelle Team-Abnahme in ISCY:

1. Docker laeuft stabil
- make docker-check ist ohne Fehler durchgelaufen.
- make docker-smoke ist ohne Fehler durchgelaufen.
2. CRA-Readiness sichtbar
- Product-Security-Scope ist gepflegt.
- SBOM-Diff und VEX-Entscheidungen sind fuer relevante Releases dokumentiert.
- Schwachstellen-/Patch-Prozess ist dokumentiert.
- Incident-/Meldeweg ist dokumentiert.
3. Cloud-Security umgesetzt
- Cloud-Services sind inventarisiert.
- Cloud-Identitaeten/Schluessel folgen Least-Privilege.
4. Windows-Hardening umgesetzt
- Baselines sind definiert.
- lokale Adminrechte, Makros, SMB und Firewall sind geregelt.
5. OT-/Produktions-Security umgesetzt
- Zonen/Conduits sind dokumentiert.
- OPC UA, MES, SCADA und Produktions-Identitaeten sind abgesichert.
6. Nachweise vorhanden
- Zu jedem Punkt gibt es in ISCY mindestens Prozess, Risiko oder Massnahme plus Evidenz.

Merksatz: Nicht nur planen - auch nachweisbar umsetzen.

6.10 Zero-Trust-Ausbaupfad:

Grundschutz wird in diesem Arbeitsstand bewusst zurueckgestellt. Der sinnvolle naechste Fokus fuer ISCY ist Zero Trust als laufende technische Posture-Sicht.

Was aktuell belastbar vorhanden ist:

- Rust-only Backend mit Zero-Trust-Webansicht unter /zero-trust/
- read-only Agent fuer Windows, macOS und Linux
- Enrollment-Token, Agent-Secret und optionale mTLS-Fingerprint-Bindung
- gefuehrter Admin-Assistent mit einmaliger, nicht cachebarer Installationsansicht
- auditierbarer Token-Lifecycle mit begrenzter Parallelverwendung und Widerruf
- persistenter Agent-State und begrenzte Offline-Queue
- administrative Agent-Secret-Rotation
- systemd-, NixOS-, Windows- und macOS-Betriebsbeispiele
- Inventar, Heartbeat und lokale OS-/MDM-/EDR-Findings
- Scores nach Device und Pillar
- offene Findings nach Severity
- Fokuskarte fuer den naechsten fachlich sinnvollen Schritt
- Flottensignale in Statusseite, JSON und Prometheus
- editierbare Agent-Policy-Profile und erwartete Coverage nach fachlichem Scope
- sichere Notification-Kanaele mit Cooldown, Delivery-Audit und periodischem Worker
- gemeinsame fachuebergreifende Notifications fuer Agent/Fleet, Evidence, Product Security/CVE, Incidents und Roadmap
- Agent-Artefaktmanifest fuer README, systemd Service/Timer/Environment-Beispiel, NixOS-Modul, Windows PowerShell Scheduled Task und macOS LaunchDaemon
- SHA-256-Pruefsummen, Signaturstatus, Verification-Status und Release-Provenance-Metadaten fuer diese Agent-Deployment-Artefakte
- Zero-Trust-Webansicht und Onboarding-Assistent mit Artefakt-, Pruefsummen-, Signatur- und Provenance-Hinweisen
- Review-Pack-Signale fuer fehlende Pruefsummen, unsignierte Agent-Artefakte, fehlende Provenance und unverifizierte Deployment-Artefakte
- tenantgebundene Rollout-Plaene mit den festen Ringen Lab, Canary, Pilot, Production und Critical
- Preflight-/Postflight-Gates, menschliche Promotion, Pause/Resume, Abbruch und operatorgefuehrte Rollback-Dokumentation ohne Remote-Ausfuehrung
- unveraenderliche kanonische Ring-Manifeste mit stabil sortierten Targets und reproduzierbarem SHA-256
- passive externe Deployment-Handoffs und kontrollierte Result-Importe mit Replay-Schutz, ohne externe Credentials oder Remote-Ausfuehrung

Agent Rollout 2.0 - Phase 1

Migration 0040rustagentrolloutgovernance ergaenzt Rollout-Plaene, kanonische Ringzeilen, explizite Targets aus vorhandenen Agent Devices, strukturierte Checks und Audit-Events. Die Weboberflaeche unter /zero-trust/rollouts/ fuehrt durch Zielversion, Scope, Ringzuordnung, Gate-Schwellenwerte und Rollback-Plan. Fremde Tenant-, Device-, Policy-, Artefakt- und Owner-Bezuege werden serverseitig abgewiesen.

Preflight bewertet vorhandene Enrollment-, OS-, Channel-, Policy-, Heartbeat-, Score-, Finding-, Artefakt-, Signatur-, mTLS- und PKI-Signale. Nach einer externen Verteilung werden das Deployment-Ergebnis und ein eigener Postflight dokumentiert. Observation und Gate-Evaluierung koennen einen Ring bestehen lassen, die naechste Stufe erfordert jedoch immer eine explizite menschliche Promotion und einen getrennten Ringstart.

Pause, Resume, Abbruch, Rollback-Anforderung und Rollback-Abschluss sind auditierte Statuswechsel. ISCY fuehrt den Rollback nicht technisch aus und speichert keine Befehle, Rohlogs, Agent-Secrets oder Enrollment-Tokens in den Rollout-Tabellen. Es gibt keine Remote-Installation, Agent-Befehlswarteschlange, automatische Softwareverteilung oder automatische Promotion.

Neue Management-Reviews frieren nur sichere Rollout-Aggregate ein; bestehende Snapshots bleiben unveraendert. Die Betriebszentrale und Prometheus zeigen aktive und pausierte Rollouts, Rollback-Pflichten, blockierte Ringe und fehlgeschlagene Targets ohne Tenantnamen oder hochkardinale Labels. Diese Governance-Unterstuetzung garantiert weder fehlerfreie produktive Rollouts noch eine Produktions-SLO und umfasst keine Wazuh-, IOC-, Behavioral-Detection- oder automatische Threat-Modeling-Funktion.

Agent Rollout 2.0 - Phase 2

Migration 0041rustagentrolloutmanifesthandoff erweitert die bestehende

Rollout-Fallakte um unveraenderliche, versionierte Ring-Manifeste. Eine Manifest-Vorschau persistiert nichts. Nach bestandenem Preflight kann nur eine Admin-Rolle das Manifest mit zweiter Bestaetigung einfrieren; vor dem Ringstart darf eine neue Version das bisherige Manifest kontrolliert abloesen. Nach dem Start ist ein erneuter Freeze gesperrt.

Das Manifest ist kanonisches kompaktes UTF-8-JSON aus kontrollierten Rust-Strukturen. Targets werden nach stabiler Device-ID und Target-ID sortiert. Eingefroren werden Ringposition, Zielversion, Deployment-Channel, Artefakt-SHA-256 und vorhandene Provenance-/Signaturstatus, Policy-Revision, sichere Plattform-/Architektur- und PKI-/mTLS-Metadaten sowie Preflight-Aggregate. Der SHA-256 bezieht sich exakt auf die exportierten Manifest-Bytes. Vor dem Ringstart verifiziert ISCY Hash, Tenant-/Ring-Scope, Artefakt, Policy und Zielmenge erneut und veraendert bei einem Blocker keinen Rollout-Status.

Ein passiver Handoff verweist auf genau ein aktives Manifest. Das sichere JSON-Uebergabepaket enthaelt Manifest-ID und -SHA-256, Rollout-/Ringbezug, Artefakt-/Policy-Snapshot und sortierte Targets. ISCY speichert keine externen Credentials, ruft keine URLs auf und uebertraegt weder Pakete noch Befehle. Vorbereitung, Export, Bestaetigung, Abschluss und Invalidierung sind serverseitig begrenzte, auditierte Statuswechsel.

Result-Pakete sind auf 1 MiB und 500 Target-Ergebnisse begrenzt. Vor der Transaktion prueft ISCY Schema, Zeitfenster, Tenant, Manifest-/Handoff-Bezug, Payload-/Manifest-Hash, bekannte Targets und Device-Referenzen, Statuswerte, Reason-Codes und Textgrenzen. Gleiche externe Batch-ID plus gleicher Payload-Hash ist idempotent; ein abweichender Hash oder ein widerspruechliches Target ergibt 409 Conflict. Die Uebernahme verwendet dieselbe interne Deployment-Result-Logik wie Phase 1 und schreibt Targetstatus, Checks, Aggregate, Import-Provenance und Audit gemeinsam. Der vollstaendige Roh-Request wird nicht gespeichert.

Die Rollout-Detailseite zeigt Manifeste, sichere Exporte, Handoffs, Importhistorie, fehlende Rueckmeldungen, Fehler und Versionsabweichungen. Management-/Regulatory-Review-Snapshots und niedrig-kardinale Operations-/Prometheus-Signale nehmen dieselben sicheren Aggregate auf. Bestehende Snapshots bleiben eingefroren. Phase 2 ist keine MDM-/RMM- oder Command-and-Control-Funktion, keine automatische Softwareverteilung und keine automatische Promotion.

Was als naechstes fachlich am meisten bringt:

1. Findings mit Risiken und Evidenzen verbinden
Aus wiederkehrenden High-/Critical-Findings sollten Risiken, Massnahmen und Evidenzanforderungen ableitbar sein.
2. MDM-/EDR-Integrationen vorbereiten
Intune, Jamf, Microsoft Defender, Wazuh, CrowdStrike oder SentinelOne sollten zunaechst als Import-/Connector-Schicht angebunden werden, nicht als Fernsteuerung.
3. Softwareinventar und CVE-Korrelation ergaenzen
Agent- oder MDM-Inventar sollte mit dem CVE-Bereich verbunden werden, damit betroffene Systeme schneller sichtbar sind.
4. Ausnahme- und Ablaufdatum erzwingen
Akzeptierte Abweichungen sollten Owner, Begruendung, Laufzeit und Wiedervorlage haben.
5. Produktive Signierung getrennt nachziehen
Das Manifest und die Provenance-Metadaten sind vorbereitet; echte Windows MSI/Intune-, macOS PKG/Jamf- und Linux-Paket-Signaturen brauchen einen separaten Schluesselmanagement- und Release-Prozess.
6. CA-/PKI-Anbindung separat planen
Private Schluessel und CSR sollen lokal auf dem Agent entstehen; Ausstellung, Rotation und Widerruf gehoeren in einen eigenen, providerunabhaengigen Meilenstein.
7. Remediation getrennt halten
Automatische Aenderungen am Endgeraet sollten erst spaeter als signierter, auditierbarer Policy-Schritt kommen.

Fachliches Kurzurteil:
ISCY ist fuer Zero Trust jetzt gut positioniert. Sollabdeckung, aktive Policy-Benachrichtigungen, gefuehrtes Onboarding, das Agent-Artefakt-/Provenance-Modell und kontrollierte Rollout-Governance sind vorhanden; die naechste Reife entsteht durch Nachweisverknuepfung, Connectoren und echte produktive Signierprozesse.

Agent-Artefakte, Pruefsummen und Release-Provenance

ISCY modelliert die vorhandenen Agent-Deployment-Artefakte als tenantgebundenes Manifest. Erfasst werden Artefaktname, Typ, Zielplattform, Paketformat, Version, Build-Profil, Commit-/Branch-Bezug, SHA-256, Groesse, Content-Type, sichere Repo-Referenz, Signaturstatus, Verification-Status, Provenance-Status, bekannte Limitierungen und technische Metadaten.

Die Plattform berechnet SHA-256-Pruefsummen nur aus einer festen Allowlist der im Repository vorhandenen Artefakte. Requests koennen keine beliebigen lokalen Pfade angeben. API, UI, Audit und Review-Pakete zeigen keine absoluten Buildpfade, keine Rohdateien, keine Tokens, keine privaten Schluessel und keine Zertifikat-Private-Keys.

Der Signaturbereich ist bewusst ein Vorbereitungsmodell. Sichtbare Statuswerte wie unsigned, notconfigured, signaturepresent, verified, failed, expired, untrusted und keymissing erlauben spaetere produktive Signierprozesse, ohne heute eine echte Produktionssignatur vorzutaueschen. Dieses Artefakt- und Signaturmodell enthaelt keine echten Code-Signing-Zertifikate, keine privaten Schluessel, keine externe PKI/CA, keine produktive CA-/CSR-Operation, keine Sigstore-/Rekor-/Fulcio-Netzwerkaufrufe und keine GitHub-Release-Veroeffentlichung.

Die wichtigsten API-Pfade:

GET /api/v1/agents/artifacts
GET /api/v1/agents/artifacts/{artifact_id}
POST /api/v1/agents/artifacts/refresh
POST /api/v1/agents/artifacts/{artifact_id}/verify-checksum
POST /api/v1/agents/artifacts/{artifact_id}/verify-signature
GET /api/v1/agents/artifacts/{artifact_id}/provenance
GET /api/v1/agents/release-provenance
GET /api/v1/agents/release-provenance/{provenance_id}
GET /api/v1/agents/onboarding/artifacts

Read-only-Rollen duerfen sichere Artefakt- und Provenance-Metadaten lesen. Schreibende Rollen duerfen Manifest-Refresh sowie Checksum- und Signaturstatuspruefungen ausloesen. Jede Aktion bleibt tenantgebunden und erzeugt ein Verification-/Audit-Ereignis ohne Secrets, Rohdateien, Authorization-Header, SQL-Details oder fremde Tenant-IDs.

Management-/Regulatory-Review-Pakete fuer NIS2, DORA, DSGVO und generische Governance nehmen diese Signale als Supply-Chain-Gaps auf. Das ist Nachweis- und Governance-Unterstuetzung, keine Rechtsberatung, keine Zertifizierung und keine automatische Behoerdenmeldung.

Agent-PKI, CSR und mTLS-Governance

ISCY fuehrt fuer Agenten eine vorbereitete CA-/PKI-/CSR-Schicht als Governance- und Statusmodell ein. Migration 0037rustagentpkicsrgovernance ergaenzt tenantgebundene Tabellen fuer PKI-Provider, CSR-/Certificate-Requests, Zertifikatsstatus und PKI-Auditereignisse. Erfasst werden CA-Provider-ID, Provider-Typ und -Status, Trust Domain, Issuing Policy, erlaubte Agent-Profile, Zertifikatslaufzeit, Renewal-Fenster, Widerrufsmodus, CRL-/OCSP-Referenz, Key-Storage-Policy, Secret-Referenzstatus und bekannte Limitierungen.

CSR-Datensaetze enthalten sichere Metadaten wie Agent-Bezug, optionalen Asset-Bezug, Common Name, SAN-Zusammenfassung, Key-Algorithmus, beantragte Nutzung, Fingerprints, Hashes, Review-Status, Freigabe, Ablehnung und Audit-Summary. Zertifikatsstatus erfasst Subject-/Issuer-Summary, Fingerprint, Laufzeit, Zertifikatsstatus, mTLS-Bindung, Rotation, Widerruf und Evidence-IDs.

Die Weboberflaeche /zero-trust/ zeigt Provider, offene CSR, Zertifikatsstatus, mTLS-Gaps sowie Rotation/Widerruf und bietet schreibenden Rollen Formulare und Review-Aktionen. Der gefuehrte Onboarding-Assistent zeigt dieselben Signale als Vorbereitungscheck. API-Pfade liegen unter /api/v1/agents/pki, /api/v1/agents/{agentid}/pki und /api/v1/agents/onboarding/pki.

Diese Stufe erzeugt keine echten Zertifikate. Sie enthaelt keine produktive CA, keine privaten Schluessel, keine echten CA-Secrets, keine externen CA- oder Sigstore-Netzwerkaufrufe, keine automatische mTLS-Aktivierung und keine produktive Rotation oder Widerrufsausfuehrung. Management-/Regulatory-Review-Pakete nehmen PKI-/CSR-/mTLS-/Rotations-/Widerrufsluecken als eingefrorene Governance-Signale auf.

6.11 Fachuebergreifende Notifications:

ISCY erweitert den vorhandenen Agent-Notification-Kanalbetrieb, statt eine zweite Notification-Engine einzufuehren. Dieselben Webhook-Kanaele,

Production-Allowlist, Redirect-Sperre, Bearer-/HMAC-Secret-Referenzen, Retry-Grenzen, Cooldowns, der periodische Worker und die manuelle Auswertung verarbeiten jetzt auch:

- abgelaufene und innerhalb von 30 Tagen ablaufende Evidence
- Evidence ohne Review, Retention oder SHA-256 bei vorhandenem Dateiobjekt
- offene und ueberfaellige CVE-Reviews, kritische offene CVEs sowie offene CVE-Korrelationsreviews
- offene Nicht-Meldeentscheidungen und fehlende Begruendungen bei Incidents
- heute faellige, ueberfaellige, blockierte oder kritische offene Roadmap-Tasks

Die Auswertung nutzt ausschliesslich vorhandene fachliche Status-, Review-, Severity- und Faelligkeitsfelder. Eine ueberfaellige Incident-Review-Frist wird nicht erfunden, weil dafuer derzeit kein eigenes fachliches Fristfeld existiert.

Ein stabiler Signal-Key kombiniert Tenant, Domaene, Objekttyp, Objekt-ID, Signaltyp und Zustands- oder Faelligkeitskontext. Nach einem Zustellversuch gilt der Cooldown des Kanals auch bei einem Fehler; interne Retry-Versuche bleiben auf den bestehenden kleinen Satz transienter Netzwerk- und HTTP-Fehler begrenzt. Migration 0029rustcrossdomainnotifications ergaenzt bestehende Delivery-Zeilen additiv um Domaene, Objektbezug, Signaltyp, sichere Fehlerklasse, letzten Versuch und naechsten moeglichen Zustellzeitpunkt.

Webhook-Payloads enthalten nur Tenant, Domaene, Objekttyp/-ID, Signaltyp, Schweregrad, Status und optionalen Faelligkeitskontext. Token, Bearer-/HMAC-Secrets, Authorization Header, personenbezogene Details, vertrauliche Objekttexte, vollstaendige Fachpayloads und interne Fehler werden weder in Responses noch in der Read-only-Ansicht ausgegeben. Administratoren duerfen Kanaele und Signalbereiche aendern; authentifizierte Read-only-Rollen sehen nur sichere tenantgebundene Delivery-Metadaten.

Bewusst nicht Teil dieses Notification-Blocks sind produktive Cloud-Credentials, produktive CA-/PKI-/CSR-Ausstellung, signierte Agent-Pakete, Release-Provenance sowie Performance-, HA- und visuelle Regressionserweiterungen.

6.12 Evidence Integrity & Disposition Phase 1:

ISCY fuehrt Evidence-Integritaet als Governance- und Audit-Funktion, nicht als neue Evidence-Engine. Migration 0033rustevidenceintegritydisposition erweitert bestehende Evidence-Datensaetze additiv um Integritaets-, Legal-Hold- und Disposition-Metadaten sowie um eine tenantgebundene Ereignistabelle fuer Integritaets- und Disposition-Ereignisse.

Die Webansicht /evidence/integrity/ zeigt berechtigten Nutzern sichere Metadaten je Evidence Item:

- Evidence-ID, Titel, Typ, Owner und Quality-Status
- letzter Integritaetscheck und Integritaetsstatus
- ob ein gespeicherter Hash vorhanden ist
- ob eine Integritaetsabweichung erkannt wurde
- Legal-Hold-Status
- Retention- und Disposition-Faelligkeit
- Disposition-Status

Die API stellt dieselben Grenzen maschinenlesbar bereit:

- GET /api/v1/evidence/integrity
- POST /api/v1/evidence/{evidenceid}/integrity-check
- POST /api/v1/evidence/integrity-checks
- GET /api/v1/evidence/{evidenceid}/integrity-events
- POST /api/v1/evidence/{evidenceid}/legal-hold
- POST /api/v1/evidence/{evidenceid}/legal-hold/release
- POST /api/v1/evidence/{evidenceid}/disposition

Admin- und Editor-Rollen duerfen Integritaetschecks ausloesen, Legal Holds setzen oder freigeben und Disposition-Entscheidungen dokumentieren. Read-only-Rollen sehen nur sichere Metadaten und Integritaetsereignisse. Alle Operationen bleiben in der Datenbankabfrage tenantgebunden; fremde Evidence-IDs werden nicht aufgeloest.

Die Re-Hash-Pruefung berechnet SHA-256 serverseitig aus dem gespeicherten

Evidence-Artefakt und vergleicht den Wert mit dem vorhandenen Evidence-Hash. Moegliche Integritaetsstatuswerte sind notchecked, valid, mismatch, missingartifact, checkfailed, quarantined und acceptedwithexception. Bei Mismatch setzt ISCY einen Review-/Quarantaene-Status und protokolliert ein sicheres Integritaetsereignis. Clientseitige Hashwerte werden fuer diese Pruefung nicht ungeprueft uebernommen.

Legal Hold wird als Governance-Status mit Grund, Akteurreferenz und Zeitstempel gefuehrt. Ein aktiver Legal Hold blockiert Disposition-Entscheidungen und setzt den Disposition-Status auf blockedbylegalhold, wenn eine Entscheidung sonst zur Freigabe fuehren wuerde. ISCY formuliert daraus keine Rechtsberatung und trifft keine automatische rechtliche Bewertung.

Disposition bleibt in Phase 1 ausdruecklich ein Metadaten- und Entscheidungsworkflow. Statuswerte wie notdue, due, reviewrequired, approvedfordisposition, dispositiondeferred und dispositioncompletedmetadataonly dokumentieren den Governance-Stand. Der Status dispositioncompletedmetadataonly bedeutet: Entscheidung und Audit sind dokumentiert, aber keine Evidence-Datei wird physisch geloescht und keine technische Datenvernichtung wird durchgefuehrt.

Auditereignisse werden fuer gestartete und abgeschlossene Integritaetschecks, gueltige Hashes, Mismatches, fehlende Artefakte, Check-Fehler, Legal-Hold-Set/Release sowie Disposition-Review, -Blockade und -Entscheidung geschrieben. Auditdetails enthalten keine Secrets, Tokens, Authorization Header, vollstaendigen Evidence-Payloads, vertraulichen Dateiinhalte, SQL-Details, fremden Tenant-IDs oder unnoetige absolute Dateipfade.

Bewusst nicht Teil dieser Phase 1 waren ein periodischer Re-Hash-Scheduler, Objektspeicher/S3, kontrollierte physische Loeschung, technische Datenvernichtung, eine neue Evidence-, Risk-, Control- oder Notification-Engine sowie Plattform-, Docker-, Postgres-, nginx-, Nix- oder Dependency-Upgrades.

6.13 Evidence Object Storage & Restore Drill Phase 2:

Phase 2 fuehrt eine interne Storage-Abstraktion fuer Evidence-Artefakte ein, ohne produktives S3, Cloud-Credentials oder ein neues Speichersystem einzufuehren. Das erste Backend ist bewusst nur localfilesystem und nutzt weiter den bestehenden Media Root. Bestehende Uploads und gespeicherte Evidence-Pfade bleiben kompatibel.

Die Storage-Schicht kapselt:

- sichere Artefaktreferenzen
- canonical path containment unterhalb des Media Root
- Blockade von absoluten Pfaden, Directory Traversal und Symlink-Flucht
- sichere Metadaten wie Backend, Referenz vorhanden, Artefakt vorhanden, lesbar, leer und Groesse
- serverseitige SHA-256-Berechnung fuer Drills
- sichere Fehlerklassen ohne absolute Pfade

Die API ergaenzt:

- GET /api/v1/evidence/storage
- GET /api/v1/evidence/{evidenceid}/storage
- POST /api/v1/evidence/{evidenceid}/storage-drill
- POST /api/v1/evidence/storage-drills
- GET /api/v1/evidence/{evidenceid}/storage-events

Der Storage-/Restore-Drill prueft, ob der Evidence-Datensatz tenantgebunden existiert, ob eine Artefaktreferenz vorhanden ist, ob das lokale Artefakt sicher unterhalb des Media Root erreichbar und lesbar ist und ob der serverseitig berechnete SHA-256 mit dem gespeicherten Evidence-Hash uebereinstimmt. Fehlende Artefakte, unsichere Referenzen, fehlende erwartete Hashes und Hash-Mismatches werden mit sicheren Fehlerklassen klassifiziert.

Die Webansicht /evidence/integrity/ zeigt Storage-Backend, Artefaktstatus, sichere Fehlerklasse, letzten Drill-/Integritaetszeitpunkt und den vorhandenen Integrity-, Legal-Hold- und Disposition-Kontext. Admin- und Editor-Rollen duerfen Storage-Drills ausloesen. Read-only-Rollen sehen nur sichere Metadaten.

Auditereignisse nutzen die bestehende Evidence-Integrity-Ereignistabelle und werden als storage Events geschrieben, darunter Start, Artefakt gefunden, Artefakt fehlt, Artefakt nicht lesbar, Hash gueltig, Hash-Mismatch, Drill fehlgeschlagen und Drill abgeschlossen. Auditdetails enthalten keine Secrets, Tokens, Authorization Header, Rohpayloads, vertraulichen Dateiinhalte, SQL-Details, fremden Tenant-IDs oder absoluten Dateipfade.

Fuer Phase 2 ist keine neue Datenbankmigration noetig: Die bestehenden Integrity-Felder aus Migration 0033rustevidenceintegritydisposition speichern letzten Pruefzeitpunkt, Status, berechneten Hash und sichere Fehlerklasse bereits ausreichend. Storage-Events werden in der bestehenden Ereignistabelle auditierbar gefuehrt.

Bewusst nicht Teil dieser Phase 2 waren produktives S3/Object Storage, Cloud-Credentials, physische Loeschung, technische Datenvernichtung, Backup-Restore aus externem Storage, komplexer Scheduler, neue Evidence-/Risk-/Control-/Notification-Engines sowie Docker-, Postgres-, nginx-, Nix- oder Dependency-Upgrades.

6.14 Evidence-Worker, kontrollierte Disposition und Object-Storage-Vorbereitung:

Der aktuelle Unreleased-Stand vertieft die Evidence-Integrity- und Storage-Grundlagen mit Migration 0035rustevidenceworkerdispositionstorage. Die Migration ist additiv und ergaenzt Worker-Laufhistorie, vorbereitete Storage-Backend-Statusdaten sowie Ausfuehrungsmetadaten fuer kontrollierte physische Disposition. Bestehende Evidence-Daten werden dabei nicht geloescht oder umgeschrieben.

Der Integritaets-Worker ist bewusst begrenzt:

- ISCYEVIDENCEWORKERENABLED schaltet den Worker-Betrieb logisch ein oder aus
- ISCYEVIDENCEWORKERBATCHSIZE begrenzt die Anzahl pro Lauf
- ISCYEVIDENCEWORKERMAXRUNTIMESECONDS begrenzt die Laufzeit
- ISCYEVIDENCEWORKERCOOLDOWNSECONDS dokumentiert den empfohlenen Mindestabstand
- Dry-Run-Laeufe dokumentieren die Auswahl, ohne Hash-Pruefungen auszufuehren

Die API stellt dafuer bereit:

- GET /api/v1/evidence/integrity/worker
- POST /api/v1/evidence/integrity/worker/run
- GET /api/v1/evidence/integrity/worker/runs

Read-only-Rollen duerfen Worker-Status und Laufhistorie lesen. Admin- und Editor-Rollen duerfen manuelle begrenzte Worker-Laeufe starten. Jeder Lauf speichert Status, Trigger, Batch-Grenzen, Runtime-Grenze, Dry-Run, gepruefte Items und Ergebniszaehler. Responses und Auditdetails enthalten keine absoluten Dateipfade, Dateiinhalte, SQL-Details, Tokens oder Secrets.

Kontrollierte physische Disposition ist vom metadata-only Status getrennt. ISCY loescht keine Datei nur wegen eines Ablaufdatums. Eine physische Aussonderung wird nur ausgefuehrt, wenn:

- dieselbe Evidence-ID im aktuellen Tenant gefunden wird
- eine schreibende Rolle handelt
- approvedfordisposition gesetzt ist
- eine dokumentierte Begruendung vorhanden ist
- kein aktiver Legal Hold und keine Aufbewahrungssperre blockiert
- die Storage-Operation ueber die gepruefte Storage-Abstraktion laeuft

Die API stellt dafuer bereit:

- GET /api/v1/evidence/disposition/candidates
- POST /api/v1/evidence/{evidenceid}/disposition/preview
- POST /api/v1/evidence/{evidenceid}/disposition/approve
- POST /api/v1/evidence/{evidenceid}/disposition/execute
- POST /api/v1/evidence/{evidenceid}/disposition/cancel
- GET /api/v1/evidence/{evidenceid}/disposition/events

Preview loescht nichts und zeigt nur sichere Metadaten wie Retention, Disposition-Faelligkeit, Legal-Hold-Status, Artefakt vorhanden ja/nein, Backend und sichere Fehlerklasse. Execute prueft Approval, Grund und Legal Hold vor jeder Storage-Operation. Wird eine Ausfuehrung verweigert, wird dies als dispositionexecutiondenied auditierbar dokumentiert, ohne das Artefakt zu

beruehren. Bei erfolgreicher Aussonderung wird die lokale Datei ueber das Filesystem-Backend entfernt, die Evidence-Dateireferenz aus dem Datensatz genommen und ein Tombstone mit Backend, sicherer Fehlerklasse, vorherigem Artefaktstatus, Hash und pathsexposed=false gespeichert. Bei Storage-Fehlern wird dispositionfailed mit sicherer Fehlerklasse dokumentiert.

Das vorbereitete Object-Storage-Modell bleibt bewusst ohne produktive Cloud-Anbindung. localfilesystem bleibt Default. s3compatible wird nur als Konfigurations- und Statussignal validiert:

- ISCYEVIDENCESTORAGEBACKEND
- ISCYEVIDENCEOBJECTSTORAGEENDPOINT
- ISCYEVIDENCEOBJECTSTORAGEBUCKET
- ISCYEVIDENCEOBJECTSTORAGEREGION
- ISCYEVIDENCEOBJECTSTORAGEACCESSKEYFILE
- ISCYEVIDENCEOBJECTSTORAGESECRETKEYFILE
- ISCYEVIDENCEOBJECTSTORAGECREDENTIALSREF

GET /api/v1/evidence/storage/backends zeigt, ob ein Backend aktiv, konfiguriert oder mit sicherer Fehlerklasse nicht bereit ist. Endpoint und Bucket werden nur als Statusmerkmale validiert; ISCY fuehrt in diesem Stand keine externen Netzwerkaufrufe aus und speichert oder zeigt keine Secretwerte.

Die Webansicht /evidence/integrity/ buendelt nun Nachweis-Integritaet, Integritaets-Worker, Storage-Backend-Status, Disposition-Kandidaten, Legal Hold, Storage-/Restore-Drills und Disposition-Aktionen in einer deutschsprachigen Operations-Ansicht. Read-only-Rollen sehen sichere Inhalte, Admin/Editor duerfen Worker starten, Dispositionen freigeben und nach Freigabe kontrolliert aussondern.

NIS2-, DORA-, DSGVO- und generische Review-Pakete frieren die neuen Evidence-Operations-Signale in Snapshots ein. Dazu gehoeren nicht gepruefte Evidence, Integritaetsabweichungen, fehlende Worker-Laeufe, offene Storage-/Restore-Pruefungen, Legal-Hold-Konflikte, faellige Dispositionen und fehlgeschlagene Aussonderungen. ISCY liefert damit Governance- und Nachweisvorbereitung, aber keine Rechtsberatung, Zertifizierung, automatische Behoerdenmeldung oder formale Einreichung.

6.15 Evidence Object Storage Client Phase 3:

Migration 0038rustevidenceobjectstorageclient ergaenzt ein produktionsnahes Object-Storage-Modell fuer Evidence-Artefakte, ohne echte Cloud-Credentials, Secretwerte oder externe Live-Netzwerkaufrufe in Tests, Doku oder Audit einzufuehren. localfilesystem bleibt der Default. s3compatible kann nun tenantgebunden als Backend-Metadatum gespeichert, validiert und mit Evidence verknuepft werden.

Die neue Backend-Konfiguration umfasst:

- backendid, Tenant und Backend-Typ (localfilesystem, s3compatible, disabled)
- Anzeige-Name, Status, Endpoint-Referenz, Region, Bucket und Key-Prefix
- Secret-Referenzen fuer Access Key, Secret Key und optional Session Token
- TLS-Pflicht, Path-Style-Flag, Endpoint-Policy und bekannte Limitierungen
- letzte Validierung mit Status und sicherer Fehlerklasse

Secret-Referenzen sind bewusst nur Referenzen wie env:..., file:..., vault:..., secret:... oder external:.... ISCY speichert und zeigt keine Secretwerte. Unsichere Eingaben wie Credentials in URLs, private Key-Fragmente, direkte Access-Key-Werte oder URL-artige Secretwerte werden mit sicheren Fehlerklassen abgelehnt.

Endpoint-Validierung blockiert fuer den produktiven Policy-Pfad insbesondere Credentials in URLs, unsichere Schemes, Loopback, Link-Local, private Netze, Cloud-Metadata-Services und lokale Hostnames. DNS wird in diesem Stand nicht extern aufgeloest; die Konfiguration dokumentiert deshalb, ob ein Runtime-DNS-Recheck erforderlich ist. Live-Redirect- und Netzwerkverhalten bleibt fuer einen spaeteren echten S3-Client explizit ausserhalb dieses PRs.

Object-Keys werden beim Anbinden einer Evidence validiert, danach aber nicht vollstaendig gespeichert. Persistiert werden nur redaktionelle Anzeige, SHA-256 des Keys, Backend-ID, erwarteter Hash und ein Contract-Status. Die

Validierung blockiert absolute Pfade, Windows-Drive-Pfade, NUL-Zeichen, Backslash-Traversal, .., fremde Prefixe und Object-Keys, die nicht den aktuellen Tenant und die aktuelle Evidence-ID kapseln.

Die API stellt dafuer bereit:

- GET und POST /api/v1/evidence/storage/backends
- PATCH /api/v1/evidence/storage/backends/{backendid}
- POST /api/v1/evidence/storage/backends/{backendid}/validate
- GET /api/v1/evidence/storage/backends/{backendid}/events
- GET und POST /api/v1/evidence/{evidenceid}/storage/object-reference
- POST /api/v1/evidence/{evidenceid}/storage/attach-object
- POST /api/v1/evidence/{evidenceid}/storage/verify-object
- POST /api/v1/evidence/{evidenceid}/storage/object-drill

Admin- und Editor-Rollen duerfen Backend-Metadaten speichern, validieren, Object-Referenzen anbinden und Contract-Drills starten. Read-only-Rollen lesen Status und sichere Metadaten. Fremde Tenant-Backends, fremde Evidence-IDs, manipulierte Object-Keys und unsichere Endpoint-/Secret-Referenzen werden ohne Fremddaten mit 4xx-Antworten und sicheren Fehlerklassen abgewiesen.

Der Object-Storage-Drill ist in dieser Phase ein Mock-/Contract-Drill hinter der bestehenden Storage-Abstraktion. Er kann sicher dokumentieren, ob ein Objekt laut Contract vorhanden, fehlend, unlesbar, zeitueberschritten, zugriffsverweigert, backendfehlerhaft oder hash-abweichend ist. Ergebnisse werden in Evidence-Integrity-Feldern, evidenceobjectreference und evidencestoragebackendevent festgehalten. Auditdetails enthalten keine Secretwerte, Tokens, Authorization Header, Objektinhalte, vollstaendigen Object-Keys, SQL-Details, fremden Tenant-IDs oder absolute lokalen Pfade.

Management-/Regulatory-Review-Pakete frieren zusaetzlich Object-Storage-Backends, ready-Backends, Konfigurationsfehler, Object-Referenzen, aufgezeichnete Object-Drills, offene Object-Drills und Objekt-/Hash-Gaps als aggregierte Signale ein. ISCY liefert damit Storage-Governance- und Restore-Readiness-Unterstuetzung, aber keine Zertifizierung, Rechtsberatung oder automatische formale Meldung.

Bewusst nicht Teil dieser Phase sind echte Cloud-Credentials, produktive S3-Live-Operationen, unkontrolliertes Loeschen, unbounded List-Operationen, externe Feeds, neue Evidence-/Risk-/Control-/Notification-Engines, Dependency-/Plattform-Upgrades, produktive PKI, signierte Agent-Pakete, Release-Veroeffentlichung, HA-/Performance-Ausbau und visuelle Regression.

6.16 S3-kompatibler Evidence-Storage-Runtime-Client:

Migration 0039rustevidences3runtimeclient aktiviert die in Phase 3 vorbereitete Sicherheitsgrundlage fuer echte, explizit konfigurierte S3-kompatible Object-Storage-Operationen. localfilesystem bleibt unterstuetzt. Ein Tenant kann Evidence ueber ein freigegebenes s3compatible-Backend hochladen, per HEAD pruefen, begrenzt und chunkweise lesen, per SHA-256 verifizieren und ausschliesslich ueber den kontrollierten Disposition-Workflow loeschen.

ISCY nutzt dafuer die bereits vorhandenen Rust-Abhaengigkeiten reqwest, hmac, sha2, chrono und tokio. Es wurde kein AWS-SDK und keine implizite Credential-Provider-Chain eingefuehrt. Die SigV4-Anfrage wird mit explizitem Endpoint, expliziter Region und genau den im Backend hinterlegten Secret-Referenzen erstellt. Redirects, Proxy-Autodiscovery, AWS-Profile, SSO, Home-Verzeichnis-Credentials sowie EC2-/ECS-Metadata-Credentials sind nicht aktiviert. TLS wird ueber das bestehende Rustls-Backend von reqwest bereitgestellt.

Backend-Konfigurationen akzeptieren ausschliesslich die zum jeweiligen Feld gehoerige Referenz env:ISCYEVIDENCEOBJECTSTORAGEACCESSKEY, env:ISCYEVIDENCEOBJECTSTORAGESECRETKEY oder env:ISCYEVIDENCEOBJECTSTORAGESESSIONTOKEN, jeweils wahlweise mit FILE-Suffix. Andere Prozessvariablen und freie file:-Pfade sind keine zulaessigen tenantgesteuerten Referenzen. Persistierte Werte werden vor der Netzwerkoperation erneut gegen dieselbe Grenze geprueft.

Der Betreiber kann den zugehoerigen Dateipfad ueber die jeweilige FILE-Umgebungsvariable konfigurieren. Der Dateileser prueft kanonische Pfadgrenzen

unter /run/secrets oder expliziten ISCYEVIDENCESECRETROOTS, Symlink-Flucht, regulaere Datei, Maximalgroesse von 16 KiB, NUL-/UTF-8-Inhalt und unter Unix unsichere Schreibrechte fuer Gruppe/Andere. Abschliessende Zeilenumbrueche werden im Speicher entfernt. vault:, external:, secret: und freie file:-Referenzen werden in Backend-Konfigurationen abgelehnt. Secretwerte werden weder persistiert noch in API, Audit, Logs, Screenshots oder Dokumentation ausgegeben und nach der jeweiligen Operation im Speicher ueberschrieben.

Vor jeder Netzwerkoperation validiert ISCY den Endpoint erneut und loest DNS mit einem festen Zeitlimit auf. In Production sind nur HTTPS und oeffentliche, nicht lokale Adressen erlaubt. Credentials in URLs, Loopback, Link-Local, RFC1918-/private Netze, CGNAT, .local und bekannte Metadata-Services werden blockiert. Der HTTP-Client bindet die vorab gepruefte Adresse, deaktiviert Redirects und Proxy-Autodiscovery und begrenzt Connect- und Operationszeit. Lokales HTTP-MinIO ist nur im Development-Modus zusammen mit ISCYEVIDENCEALLOWLOCALTESTENDPOINT=true erlaubt; Production kann diese Ausnahme nicht aktivieren.

Object Keys kommen nicht aus einem Upload-Request. ISCY erzeugt eine zufaellige opaque Object-ID und leitet den Key serverseitig als {prefix}/tenants/{tenantid}/evidence/{evidenceid}/objects/{objectid} ab. Die Datenbank speichert die opaque ID, den SHA-256 des kanonischen Keys und eine redigierte Anzeige. Der vollstaendige Key wird fuer jede Operation neu erzeugt und nicht in UI oder Audit geschrieben. Traversal, absolute Pfade, Backslashes, NUL, Windows-Laufwerke, Prefix-Ausbruch sowie Tenant-/Evidence-Wechsel sind damit ausgeschlossen.

Neue beziehungsweise runtime-faehige API-Pfade:

- POST /api/v1/evidence/storage/backends/{backendid}/validate-live
- GET /api/v1/evidence/storage/backends/{backendid}/runtime-status
- POST /api/v1/evidence/{evidenceid}/storage/upload
- GET /api/v1/evidence/{evidenceid}/storage/download
- POST /api/v1/evidence/{evidenceid}/storage/verify-runtime
- der bestehende POST /api/v1/evidence/uploads akzeptiert optional storagebackendid und schreibt dann keine lokale Artefaktkopie

Downloads bleiben an Authentifizierung, Tenant, Evidence-ID, Owner/Rolle und Schutzklasse gebunden und verwenden private, no-store. Admin-/Editor-Rollen duerfen Upload, Live-Pruefung und Restore-Pruefung starten. Auditoren und Read-only-Rollen sehen sichere Status-/Nachweisdaten; Runtime-Schreibaktionen sind gesperrt. Der Evidence-Worker verwendet fuer S3-Referenzen denselben begrenzten HEAD-/GET-/SHA-256-Pfad.

Die lokalen Routen GET /api/v1/evidence/{evidenceid}/download und GET /evidence/{evidenceid}/download speichern fuer bekannte tenant-eigene Objekte localdownloadallowed oder localdownloaddenied in der bestehenden evidenceintegrityevent-Historie. Das INSERT selbst ist an Tenant und Evidence-ID gebunden. Erst genau eine eingefuegte Zeile und ein erfolgreicher Transaktionscommit geben die Antwort frei. Bei fehlender Audit-Tabelle, unterdruecktem INSERT oder Commitfehler folgt HTTP 503 ohne Dateibytes. Das gilt fuer alle lokalen Schutzklassen, zusaetzlich zur bestehenden Rollen-, Owner-, Pfad- und Cache-Pruefung.

Das Ereignis dokumentiert responseprepared, keine bestaetigte Zustellung an den Client (responsedeliveryconfirmed=false). Es enthaelt nur Tenant/Evidence/Actor, Zeitpunkt, Entscheidung, festen Grund und bei Freigabe die Byteanzahl. Pfade, Dateinamen, Titel, Inhalte und Sessionwerte fehlen. Unauthentifizierte sowie fehlende/fremde Objekte erzeugen keine objektbezogene persistente Historie; ihre sicheren Runtime-Logs bleiben erhalten.

Der S3-Runtime-Download GET /api/v1/evidence/{evidenceid}/storage/download verwendet dieselbe gemeinsame tenantgebundene Commit-Pruefung. Sein bisheriger Ereignistyp storageobjectdownloaded bleibt fuer Freigaben erhalten; bekannte Ablehnungen werden als storageobjectdownloaddenied gespeichert. Rollenfehler, fehlende/unfertige Referenzen, ungueltige kanonische Keys, Backend-/GET-Fehler und Hash-Mismatches liefern keine Datei und erhalten einen festen Auditgrund. Scheitert das Audit selbst, folgt HTTP 503 statt Dateiausgabe. Ein erfolgter Remote-GET kann dadurch bereits stattgefunden haben; das Audit ist keine transaktionale Bestaetigung des Storage-Servers.

S3-Auditdaten enthalten keine Bucketnamen, Keys, Endpoints oder Secret-Referenzen.

hashverified=true bedeutet einen echten Vergleich mit vorhandenem erwartetem Hash. Bei historischen leeren Hash-Metadaten bleibt das bisherige Leseverhalten erhalten, der Auditwert ist ausdruecklich false. Hash-Mismatches bleiben gesperrt. Die Security-Middleware erhaelt private, no-store und ergaenzt max-age=0; die allgemeine restriktive Cachevorgabe fuer andere Antworten bleibt erhalten.

Remote-DELETE wird niemals als freie Object-, Prefix-, Batch- oder Bucket-Operation angeboten. Der bestehende Disposition-Execute-Pfad prueft Tenant, Evidence, Object-Referenz, dokumentierte Freigabe, Begruendung und Legal Hold vor der Remote-Operation. Danach prueft ISCY die Object-Abwesenheit, erhaelt Tombstone, opaque ID und Key-Hash und behandelt ein bereits fehlendes Object idempotent. Fehler nach Remote-Upload und vor DB-Finalisierung erzeugen orphanreviewrequired statt einer gueltigen Evidence-Referenz.

Der reproduzierbare Testpfad make object-storage-integration startet eine versionierte lokale MinIO-Instanz mit ausschliesslich Dummy-Credentials und prueft PUT, HEAD, GET, SHA-256, Hash-Mismatch, Object-missing, Access-denied, DELETE, wiederholtes DELETE und Cleanup. Der gleichnamige GitHub-CI-Job ist der massgebliche Protokolltest, falls lokal kein Docker vorhanden ist.

Management-/Regulatory-Review-Snapshots enthalten nur aggregierte S3-Runtime-Objects, Upload-Fehler, offene Restore-Pruefungen, sichere Runtime-/Endpoint-/Secret-Fehlerklassen und Orphan-Reviews. Das unterstuetzt NIS2-, DORA-, DSGVO- und generische Governance-Nachweise, ist jedoch keine Rechtsberatung, Zertifizierung oder automatische Behoerdenmeldung.

Bewusste Grenzen: kein Bucket Create/Delete, kein List Objects, keine ACL- oder Bucket-Policy-Verwaltung, keine Presigned URLs, kein Multipart Upload/Copy, keine Cloud-native Secret-Manager, keine automatische Credential Discovery, keine unkontrollierten Deletes und keine neuen Evidence-/Risk-/Control- oder Notification-Engines.

Als Preflight fuer diesen Roadmap-Block wurde Migration 0034rustsupplierproductsecuritygovernance zusaetzlich gegen eine echte lokale temporaere PostgreSQL-Instanz geprueft. Tabellen, Supplier/Product-Security-Datensatz, Statusaenderung, Evidence-Link, Events und Vertrags-/Exit-Historie liefen erfolgreich; die temporaere Instanz wurde danach gestoppt und die Daten wurden geloescht.

Bewusst nicht Teil dieses Blocks sind echte Cloud-Credentials, Cloud-native Secret-Manager, externe Live-Feeds, automatische Rechtsbewertung, automatische Behoerdenmeldung, neue Risk-/Control-/Notification-Engines, Dependency- oder Plattform-Upgrades, produktive PKI-/CSR-Ausstellung sowie Performance-, HA- oder Visual-Regression-Aussagen ausserhalb der nachfolgend beschriebenen Testgrenzen.

6.17 Performance-, HA- und visuelle Regression:

ISCY trennt jetzt Liveness, Readiness und Startup. /health/live sagt nur, dass der Rust-Prozess lebt. /health/ready prueft Datenbank, vollstaendigen Migrationsstand und aktive Request-Annahme. /health/startup zeigt lediglich eine zufaellige nicht sensitive Instanz-ID und den Startzeitpunkt. Fehler enthalten keine Connection Strings, SQL-Details, Secrets oder internen Pfade.

SIGINT und SIGTERM schalten zuerst Readiness ab. Laufende Requests duerfen begrenzt enden; der Notification-Worker beendet seinen aktuellen Zyklus und stoppt danach. PostgreSQL-Migrationen werden mit einem Advisory Lock serialisiert. SQLite bleibt ausdruecklich ein Single-Instance-Pfad.

make performance-smoke startet eine wegwerfbare Testtopologie und misst Health, typische Lesewege, Regulatory Preview, einen kleinen Write sowie den S3-Lifecycle. Der JSON-/Markdown-Bericht enthaelt p50, p95, p99, Maximum und Fehlerrate. Die grosszuegigen Grenzen sind CI-Regressionsbudgets und keine Produktions-SLOs.

In CI liegen Performance- und Visual-Rohdaten in privaten 0700-Wegwerfverzeichnissen. Vor einem Upload erzeugt ein fail-closed Sanitizer ein getrenntes, allowlist-basiertes Staging mit SHA-256-Manifest. Performance-Artefakte enthalten nur synthetische Aggregatmetriken; Visual-Artefakte nur eine minimierte synthetische Zusammenfassung und gegebenenfalls validierte Diff-PNGs. Traces, Videos, Rohscreenshots, Browserprofile, Cookies, Storage-State, Logs, Datenbanken, .env-Dateien, Zertifikate und Schluessel

sind nicht uploadfaehig. Fremde Pull-Request-Repositories und jede fehlgeschlagene Sanitization blockieren den Upload.

make ha-integration prueft zwei identische Backends mit gemeinsamem PostgreSQL 16 und MinIO hinter nginx 1.31. Daten und Evidence werden ueber unterschiedliche Instanzen geschrieben, gelesen und verifiziert. Danach wird Failover in beide Richtungen sowie ein paralleler Migrationsstart geprueft. Lokaler Dateispeicher und SQLite werden nicht als HA-faehig dargestellt.

make visual-regression vergleicht 46 bewusst versionierte Playwright-Baselines fuer 23 zentrale Seiten bei Desktop- und kleinem Laptop-Viewport. Neben Pixelabweichungen prueft die Suite leere Hauptbereiche, 500-Seiten, horizontalen Ueberlauf, abgeschnittene Tabellenueberschriften und sichtbare Secrets. CI aktualisiert Baselines nie automatisch.

Die Testarchitektur, Budgets, Baseline-Pflege und verbleibenden Single Points of Failure sind in docs/PERFORMANCEHAVISUALTESTING.md beschrieben. PostgreSQL, MinIO und nginx sind in der Testtopologie selbst jeweils Einzelinstanzen. ISCY behauptet damit weder Multi-Region-HA noch beliebige Skalierbarkeit, SLA-Erfuellung, Zertifizierung oder Rechtskonformitaet.

6.18 Finales Hardening und Release-Lifecycle:

Der letzte veroeffentlichte Stable Release ist V23.7.33. Er wurde am 16. August 2026 veroeffentlicht und ist an den unveraenderlichen Tag-/Commit-Stand 2820f19f5fa33069db81e05c10949f2558948d04 gebunden. Der zugehoerige Published-Snapshot liegt unter release/published/V23.7.33.json.

Der aktuelle main-Lifecycle arbeitet auf V23.7.34 mit dem Status developmentunreleased. Fuer V23.7.34 existiert deshalb bewusst noch kein Tag und kein GitHub Release. Ein separater Release-Prep-Schritt muss spaeter erneut Teststatus, Artefakte, Reproduzierbarkeit und menschliche Freigabe binden.

Das Root-Manifest release/release-manifest.json beschreibt aktuell unter anderem:

- Rust-Paket 0.3.22 und MSRV Rust 1.88,
- PostgreSQL 16 als Standarddatenbank,
- SQLite als Single-Instance-Kompatibilitaetspfad,
- PostgreSQL 18.4 als zusaetzlichen logischen Forward-Restore- und Anwendungskompatibilitaetspfad,
- 47 fortlaufende Migrationen (0001 bis 0047) und 46 visuelle Baselines,
- CycloneDX-1.5-SBOM,
- einen vorbereiteten, aber noch unsignierten Provenance-Vertrag.

Die vorhandenen Pflichtpruefungen laufen in make release-candidate-check zusammen. GitHub CI aggregiert elf Produkt-/Betriebsjobs fuer Secret-Scan, Rust, MSRV, Bootstrap, Nix, Object-Storage, Performance, HA, Visual Regression, Compose und das portable Linux-Binary. CodeQL fuer Actions, JavaScript/TypeScript und Rust bleibt ein separater Pflichtnachweis. Ein fehlgeschlagener Test oder eine nicht verfuegbare externe Testabhaengigkeit gilt als fehlgeschlagenes Gate und darf nicht als erfolgreiche Release-Validierung umgedeutet werden.

Die CI-/Produkt-Toolchain und der digest-gepinnte portable Release-Builder verwenden Rust 1.98.1. Die MSRV bleibt Rust 1.88; die davon getrennte Nix-Toolchain verwendet weiterhin Rust 1.95.0. Die Build-Vorgabe allein ist kein Nachweis bestandener CI-, Reproduzierbarkeits- oder Portabilitaetspruefungen. Das aktuelle flake.lock bindet nixpkgs auf da6dc0e7b26060f2f71ce3ef7088fb56ee44fa36 im Kanal nixos-26.05.

Eine reproduzierbare CycloneDX-SBOM, SHA-256-Checksummen, Release-Manifest und Reproduzierbarkeitsmetadaten sind vorhanden. Sie sind keine kryptografische Publisher-Signatur. Der aktuelle Release-Vertrag fuehrt den Signaturstatus ausdruecklich als unsigned; produktive Signierung/Attestation bleibt ein offener Hardening-Schritt.

Die Features bis einschliesslich Migration 0045rustsoftwareapprovalexceptionpolicy sind im V23.7.33-Stable-Tag enthalten. Die Migrationen 0046rustmachinerycrasafetysecurityfoundation und

0047rustsoacutoverrestoration gehoeren zum nachfolgenden V23.7.34-Development-Stand und sind noch nicht Teil des letzten Stable Releases. Migration 0047 stellt das tenantgebundene SoA-Datenmodell fuer SQLite und PostgreSQL wieder her; der separate Assessment-Wizard bleibt wie in Abschnitt 5.2 beschrieben offen.

Native Threat Intelligence, Continuous Vulnerability Intelligence, Software-Hygiene und Software Approval/Exceptions bleiben bewusst von aktiver Response getrennt. Sie installieren oder blockieren keine Software und senden keine Agentenbefehle. Ebenso ist der CVE-LLM-Pfad aktuell ein deterministischer Rust-Stub und keine echte Modellinferenz; eine RAG-Runtime ist nicht implementiert.

Der owner-kontrollierte ISCY Codex PR-Orchestrator bietet fuer autorisierte Same-Repository-Draft-PRs gegen main die Routen /iscy status, /iscy review, /iscy fix-ci und /iscy verify. Read-only Review/Verify, Workspace-begrenzte Korrekturen, Head-/Diff-Pruefungen und hoechstens zwei CI-Fix-Versuche folgen Least-Privilege- und fail-closed Grenzen. Codex erhaelt keine Merge-, Tag- oder Release-Berechtigung; die Merge-Grenze bleibt menschlich.

Historische Release-Candidate- und Maintenance-Details stehen im CHANGELOG.md, in docs/releases/ und in den Published-Snapshots. Das laufende Handbuch verwendet dagegen den aktuellen Stable-/Development-Lifecycle.

7. Was die wichtigsten Begriffe bedeuten:

- ISO 27001: internationaler Standard fuer Informationssicherheits-Managementsysteme
- ISMS: Managementsystem fuer Informationssicherheit
- NIS2: EU-Regelwerk fuer Cybersecurity-Pflichten wesentlicher und wichtiger Einrichtungen
- KRITIS: Kritische Infrastrukturen
- SoA: Statement of Applicability, also die begruendete Auswahl und Bewertung von Controls
- Audit: systematische Pruefung
- Finding: Feststellung aus einer Pruefung
- Evidence: Nachweis
- CVE: standardisierte Kennung fuer eine bekannte Schwachstelle
- CVSS: Basisscore fuer technische Schwere
- EPSS: Wahrscheinlichkeitsmodell fuer die Ausnutzung einer Schwachstelle
- KEV: Liste bekannter aktiv ausgenutzter Schwachstellen
- TARA: Threat Analysis and Risk Assessment
- PSIRT: Product Security Incident Response Team

8. Was ISCY ausdruecklich nicht ersetzt:

ISCY ersetzt nicht:

- eine Rechtsberatung
- eine vollautomatische Sicherheitsbewertung ohne Fachpruefung
- operative Patch- oder Deployment-Tools
- ein SIEM oder SOC
- ein vollwertiges Projektmanagement-Werkzeug fuer alle Unternehmensbereiche

ISCY strukturiert, dokumentiert, priorisiert und verbindet. Entscheidungen muessen trotzdem fachlich verantwortet werden.

9. Empfehlungen fuer die Einfuehrung:

- mit einem Tenant und einem realen Scope starten
- lieber wenige, aber belastbare Daten pflegen
- Begriffe intern vereinheitlichen
- Nachweise frueh sammeln
- Risiken nicht technisch, sondern geschaeftlich formulieren
- Produkt- und Schwachstellenlogik nur dort aktivieren, wo sie wirklich gebraucht wird

10. Strategische Weiterentwicklung:

Die Rust-Migration ist abgeschlossen. Mit V23.7.19 ist das regulatorische Organisationsprofil als erster strategischer Baustein umgesetzt; V23.7.20 ergaenzt Management-Review- und Audit-Pakete als steuerbaren Review-Workflow; V23.7.21 liefert Exporte, Snapshot-Ruecklinks und Evidence-Qualitaet; V23.7.22 setzt Third-Party-/Supplier-Risk als eigenes Rust-Web-/API-Modul um; V23.7.23 baut Product Security um VEX, SBOM-Diff und CRA-Readiness

aus; V23.7.24 fuegt AI Governance hinzu; V23.7.25 schliesst Agent-Policy-Profile, erwartete Flottenabdeckung und aktive Policy-Webhooks an; V23.7.26 ergaenzt versionierte Product-Security-Evidence-Pakete. Migration 0027rustaigovernancelinks verbindet AI-Systeme tenantgebunden mit Risiken, Roadmap-Tasks, Incidents und Changes. Migration 0028rustguidedagentonboarding ergaenzt den gefuehrten, tenantgebundenen Agent-Rollout mit Token-Lifecycle, Policy-Zuordnung und Auditspur. Migration 0029rustcrossdomainnotifications fuehrt Evidence-, CVE-, Incident- und Roadmap-Signale in denselben sicheren Kanalbetrieb. Migration 0031rustsupplierreviewworkflow ergaenzt Supplier-Reviews mit Freigabehistorie, Subprocessors, Vertragslaufzeiten, Exit-Test-Nachweisen und tenantgesicherten Evidence-/Control-/Risk-Links. Migration 0032rustmanagementregulatorytemplates ergaenzt Management-/Regulatory-Templates fuer ISO 27001, NIS2, DORA, DSGVO, KRITIS und generische Security-Governance-Reviews. Kontextsensitive Regulatory Review Packs fuer NIS2, DORA und DSGVO nutzen diese bestehende Snapshot-Schicht und nehmen Evidence-Integrity-/Storage-Aggregate auf, ohne ein neues Compliance-Silo oder ein zweites Evidence-System anzulegen. Migration 0033rustevidenceintegritydisposition ergaenzt Evidence Integrity & Disposition Phase 1 mit manueller und begrenzter Batch-Re-Hash-Pruefung, Legal-Hold-Metadaten, metadata-only Disposition und auditierbaren Integritaetsereignissen. Evidence Object Storage & Restore Drill Phase 2 nutzt diese bestehenden Metadaten fuer eine interne lokale Storage-Abstraktion, sichere Artefaktreferenzen und tenantgebundene Restore-/Integritaetsdrills ohne neues Speichersystem. Migration 0034rustsupplierproductsecuritygovernance verbindet Lieferanten, Produkte/Services, lokale Advisory-/PSIRT-/CVE-Metadaten, Evidence, Review-Status, Vertrags-/Exit-Plan-Historie und Regulatory Review Packs tenantgebunden, ohne externe Live-Feeds einzufuehren. Migration 0035rustevidenceworkerdispositionstorage ergaenzt begrenzte Evidence-Worker-Laeufe, kontrollierte physische Disposition mit Tombstone-Metadaten und ein vorbereitetes Object-Storage-Statusmodell ohne echte Cloud-Credentials. Migration 0036rustagentreleaseartifactprovenance ergaenzt Agent-Artefaktmanifest, SHA-256-Pruefsummen, Signaturstatus, Release-Provenance und Verification-Audit fuer vorhandene Deployment-Artefakte, ohne echte Produktionsschluessel, externe PKI/CA oder GitHub-Release-Veroeffentlichung einzufuehren. Migration 0037rustagentpkicsrgovernance ergaenzt Agent-PKI-/CSR-/mTLS-Governance als Metadata-only-Modell ohne produktive CA, private Schluessel, CA-Secrets, externe Ausstellung oder automatische mTLS-Aktivierung. Migration 0038rustevidenceobjectstorageclient ergaenzt tenantgebundene Object-Storage-Backend-Metadaten, Secret-Referenzstatus, sichere Object-Referenzen, SSRF-/Key-Validierung, Contract-Drills und Review-Pack-Signale. Migration 0039rustevidences3runtimeclient aktiviert darauf echte, begrenzte S3-kompatible Runtime-Operationen mit expliziter Secret-Aufloesung, DNS-/SSRF-Revalidierung, kanonischen Object-IDs und kontrolliertem Remote-Delete. Die weitere ISCY-Agenda konzentriert sich deshalb nicht mehr auf Abloesung alter Python-/Django-Pfade, sondern auf fachliche Produktreife.

Migration 0040rustagentrolloutgovernance ergaenzt darauf kontrollierte Rollout-Ringe, Preflight-/Postflight-Gates, menschliche Promotion und operatorgefuehrten Rollback fuer bestehende Agenten, ohne Remote-Ausfuehrung oder automatische Softwareverteilung einzufuehren.

Migration 0041rustagentrolloutmanifesthandoff ergaenzt darauf unveraenderliche Ring-Manifeste, reproduzierbare SHA-256, passive externe Deployment-Handoffs und begrenzte transaktionale Result-Importe. ISCY bleibt dabei Governance- und Evidence-Control-Plane ohne externe Credentials, Remote-Ausfuehrung oder automatische Verteilung.

Migration 0042rustnativethreatintelligenceobservations ergaenzt lokal validierte Indicators, normalisierte Referenzen auf vorhandene Agent- und Vulnerability-Findings, manuelle Indicator-Links, Triage und Audit. Der Entwicklungsstand fuehrt weder Raw-Log-Ingestion, externe Feeds, automatische Korrelation noch Incident-/Evidence-Erzeugung oder aktive Reaktion ein.

Migration 0043rustcontinuousvulnerabilityintelligence erweitert die vorhandenen globalen CVE-Referenzdaten um NVD-, CISA-KEV- und FIRST-EPSS-Provenance, persistente Feed-Laeufe und Checkpoints. Tenantgebundene Korrelationen und vorhandene Product-Security-Vulnerability-Findings erhalten erklaerbare Match-, Datenalter-, Hygiene- und Prioritaetsfelder. Neue Permissions werden keiner Bestandsrolle automatisch zugewiesen. Der Pfad erzeugt keine Security Observation, keinen Incident, keine Evidence und keine aktive Reaktion.

Migration 0044rustvulnerabilityhygienelifecycle ergaenzt dafuer tenantgebundene Evaluationslaeufe und Last-Seen-Generationen. Eine aktive Korrelation wird nur nach einem vollstaendig erfolgreichen Scope als STALE markiert, wenn sie nicht erneut beobachtet wurde. Das zugehoerige Finding bleibt bei einer anderen aktiven bestaetigten Korrelation ACTIVE, wird bei ausschliesslich unsicheren aktiven Kandidaten REVIEW und ohne aktive

Korrelation HISTORICAL. Eine spaetere belastbare Beobachtung reaktiviert dieselben Datensaetze; historische Provenance bleibt erhalten.

Query-/Kandidatenlimits, unvollstaendige Inventarquellen, Parser- oder Datenbankfehler und verlorene Leases kennzeichnen den Lauf als INCOMPLETE. Solche Laeufe duerfen keine Stale-Markierung oder Entwarnung aus blosser Nichtbeobachtung ableiten und ersetzen nicht den letzten vollstaendigen Evaluationszeitpunkt. Nicht erneut beobachtet bedeutet auch nach einem vollstaendigen Lauf nicht automatisch nicht betroffen und erzeugt keine VEX-Aussage. Manuelle Triage, Verantwortliche, Fristen, VEX, Risk Acceptance, Kommentare und Compensating Controls bleiben erhalten. /cves/ und die Software-Hygiene-API zeigen Lifecycle, Datenalter und den sicher begrenzten Laufstatus, ohne interne Lease-, Actor- oder globale Checkpointdetails an Tenantrollen offenzulegen.

Migration 0045rustsoftwareapprovalexceptionpolicy ergaenzt exakte tenantgebundene Policies, befristete Exceptions, persistierte passive Bewertungen und begrenzte Auditereignisse. PROHIBITED und RESTRICTED gewinnen deterministisch vor APPROVED; eine Ausnahme deckt nur ihre eigene Policy ab. UNMANAGED und REVIEWREQUIRED bleiben fail-closed sichtbar. Self-Approval, fremde Tenantziele, veraltete Revisionen und unbefristete Ausnahmen werden serverseitig abgewiesen. Die Funktion aendert keine Software, Finding-Triage, VEX-, Risk-Acceptance-, Incident-, Evidence-, Security-Observation- oder Agentdaten.

Migration 0046rustmachinerycrasafetysecurityfoundation ergaenzt die produktbezogene Safety-&-Conformity-Schicht. Die Migration klassifiziert kein Bestandsprodukt automatisch, uebernimmt keine Organisations-Applicability und erzeugt keine Compliance-Aussage. Requirements und Evidence werden aus den bestehenden Engines referenziert; typisierte Foreign Keys, Tenant-Pruefungen, optimistische Revisionen, SQLite-Write-Serialisierung und transaktionales Audit sichern die Objektgrenzen.

Die priorisierte Roadmap liegt in docs/ISCYSTRATEGICROADMAP.md und umfasst:

1. Supplier/Product-Security-Workflow fachlich weiter polishen, z. B. feinere Import-Vorbereitung fuer Hersteller-Advisorys, Contract-/Exit-Reifegrade und Review-Pack-Gliederung
2. Den S3-kompatiblen Evidence-Storage nach menschlicher Security-Review in einer isolierten Betreiberumgebung mit produktiven Endpoint-, Secret-Root- und Restore-Vorgaben pilotieren
3. Produktive Agent-Paketsignierung und eine spaetere produktive CA-/PKI-Stufe auf Basis des vorbereiteten Artefakt-/Provenance- und PKI-/CSR-Governance-Modells
4. Native Threat Intelligence nach menschlicher Review erst in getrennten Meilensteinen um kontrollierte Connectoren oder automatische Vorschlaege erweitern; die Phase-1-Grenzen bleiben bis dahin verbindlich

Der Leitgedanke bleibt: ISCY soll keine Regulierungen als Silos verwalten, sondern Organisation, Assets, Suppliers, Produkte, Controls, Risiken, Evidence, Incidents, Product Security, AI Governance, Threat Intelligence, Security Observations, Agent-Posture und Roadmap-Arbeit in einem gemeinsamen Steuerungsmodell verbinden.

11. Git-Bezug dieses Handbuchs:

Dieses Handbuch ist bewusst als Markdown-Datei im Repository abgelegt, damit es:

- versioniert werden kann
- mit dem Produkt mitwaechst
- in Pull Requests geprueft werden kann

Das PDF-Handbuch docs/ISCYHandbuch.pdf wird reproduzierbar aus Markdown erzeugt:

make docs-pdf

Nach einer bewussten Handbuchaenderung werden zuerst Markdown und erzeugtes PDF inhaltlich geprueft. Anschliessend aktualisiert der explizite, auf unveroeffentlichte Staende begrenzte Abgleich das Root-Manifest und release/SHA256SUMS transaktional:

./scripts/update_release_dependency_metadata.sh --include-handbook

Ohne diese Option aktualisiert der Dependency-Abgleich den Handbuchhash nicht stillschweigend. Der PDF-Snapshot-Test vergleicht das versionierte PDF bytegenau

mit der Ausgabe des aktuellen Generators aus diesem Markdown. Wiederholte Erzeugung darf keine Drift erzeugen. Historische Releases, Tags, Published-Snapshots und der freigegebene SBOM-Snapshot bleiben unveraendert.